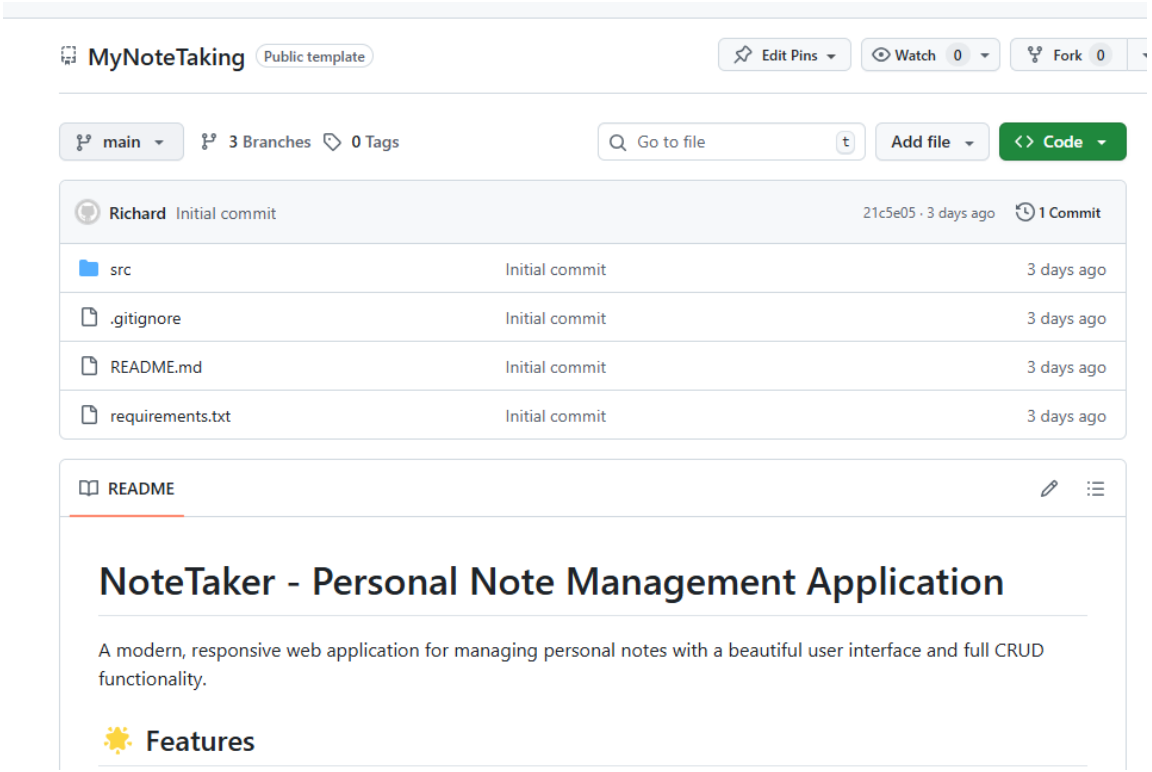
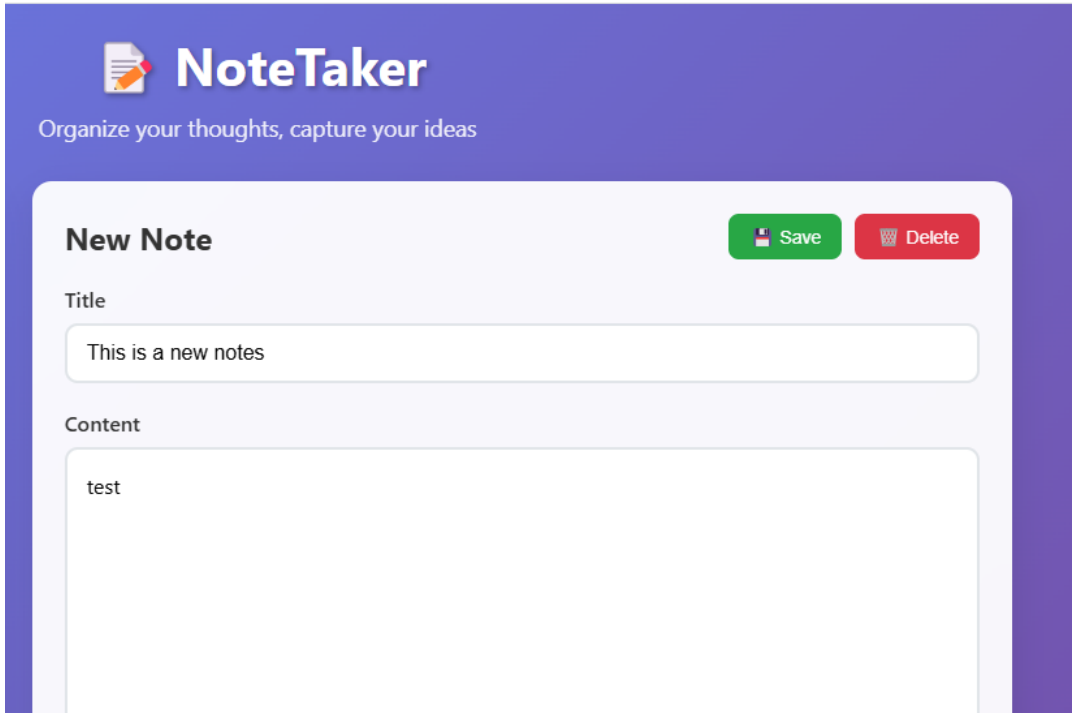
Lab: AI Assisted Coding

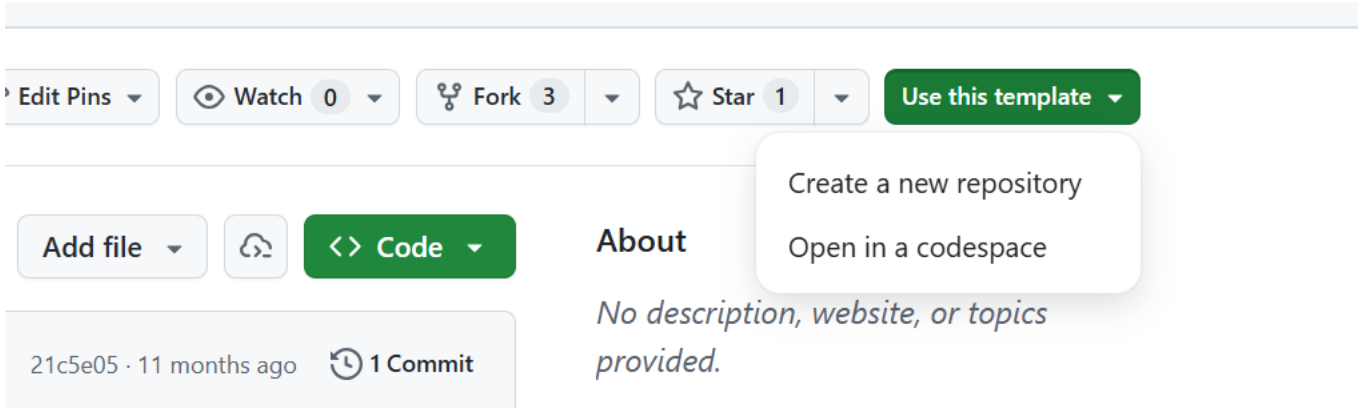
Overview





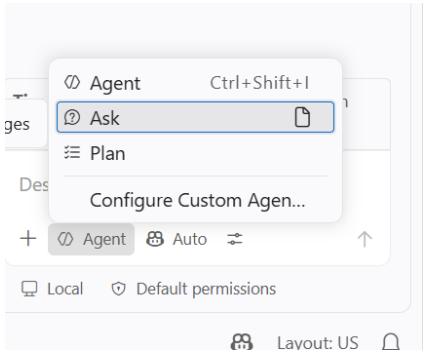
Github Repository: https://github.com/HKPolyUSE/MyNoteTaking
The note taking app developed by Manus AI: https://3dhkilc88dkk.manus.space/

- Approach 1:
  - Use git to clone this repository to your local computer and work locally
- Approach 2:
  - Use this template -> Create a new repository. Create codespaces from the repository
- You may use Github Copilot, Opencode, other coding agents to complete this exercise

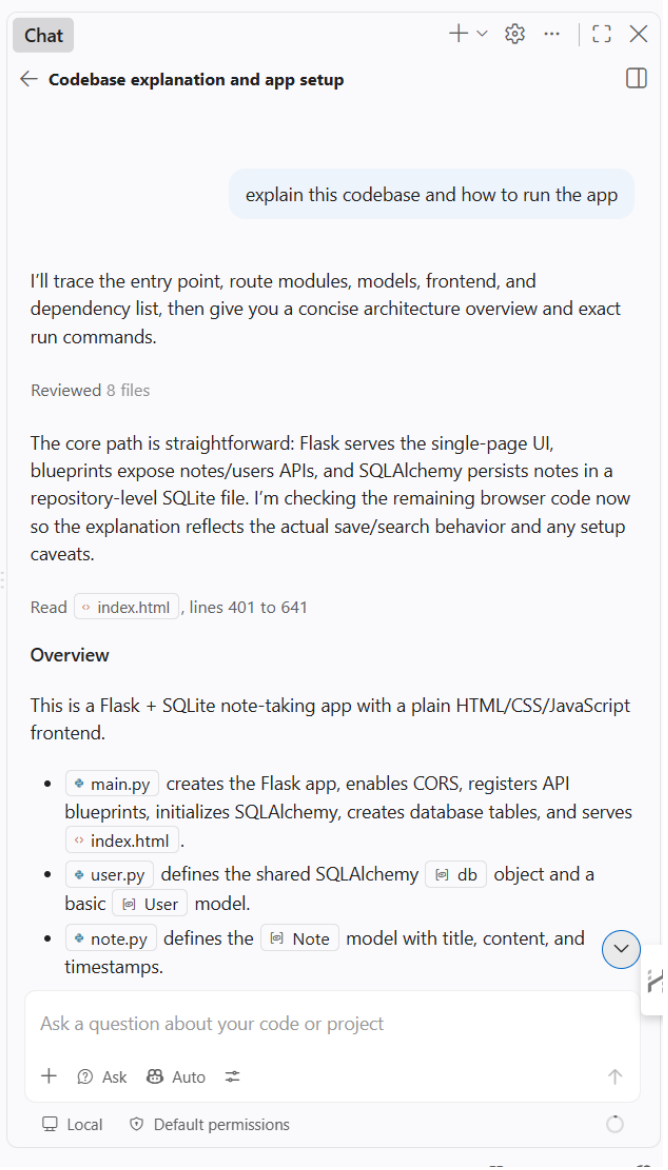


Explore the codebase

- Use **Ask** mode in GitHub Copilot
- Follow the suggested steps to run the app

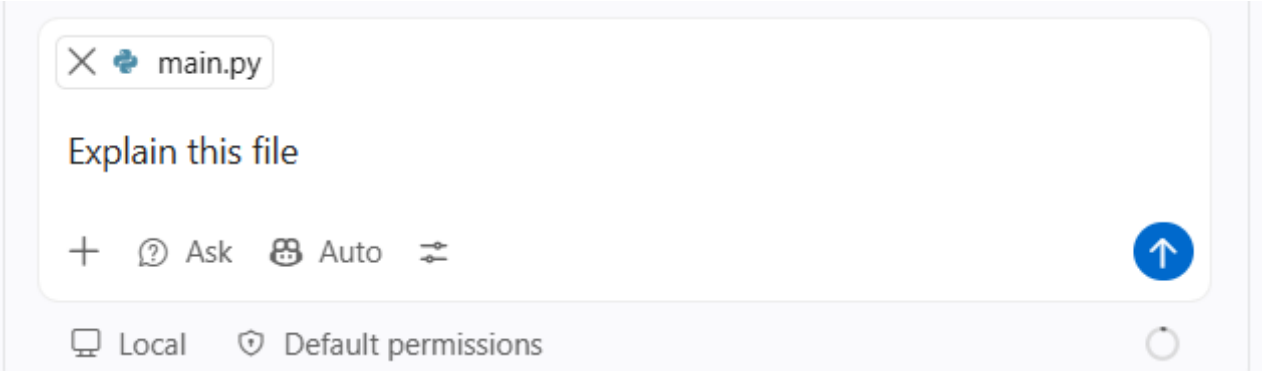


Ask	**Conversational Assistance** The standard chat interface where you ask questions, request explanations, or get code snippets without modifying files directly.	• Understanding existing code • Debugging concepts • Generating small snippets to copy-paste
Plan	**Architecture & Strategy** Analyzes your request and codebase to build a step-by-step implementation plan (blueprint) without writing code immediately.	• Breaking down complex features • clarifying requirements before coding • Designing large refactors
Agent	**Autonomous Execution** An autonomous mode that can plan, edit, run terminal commands, and fix errors across multiple files to complete a task from start to finish.	• End-to-end feature implementation • Complex multi-file tasks • Tasks requiring iterative testing and fixing



Add to context

- VS Code automatically adds the active file as a context item in the chat input box
- E.g. Suppose you select main.py and view it in editor
- You can drag additional files to the chat areas and use them as the context.
- # mentions
  - # followed by the file name, folder name, or symbol name



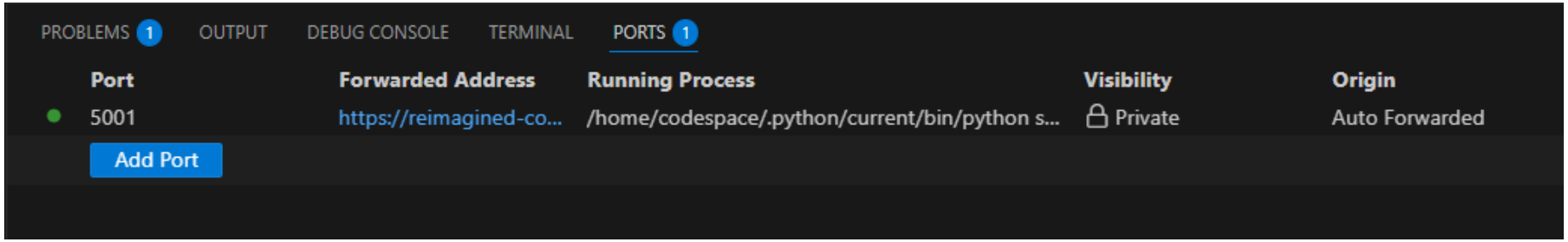
Starting the app

```
PS C:\programs\notetaking-app\src> python main.py
 * Serving Flask app 'main'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use
a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5001
 * Running on http://158.132.10.133:5001
Press CTRL+C to quit
 * Restarting with watchdog (windowsapi)
 * Debugger is active!
 * Debugger PIN: 296-050-124
```

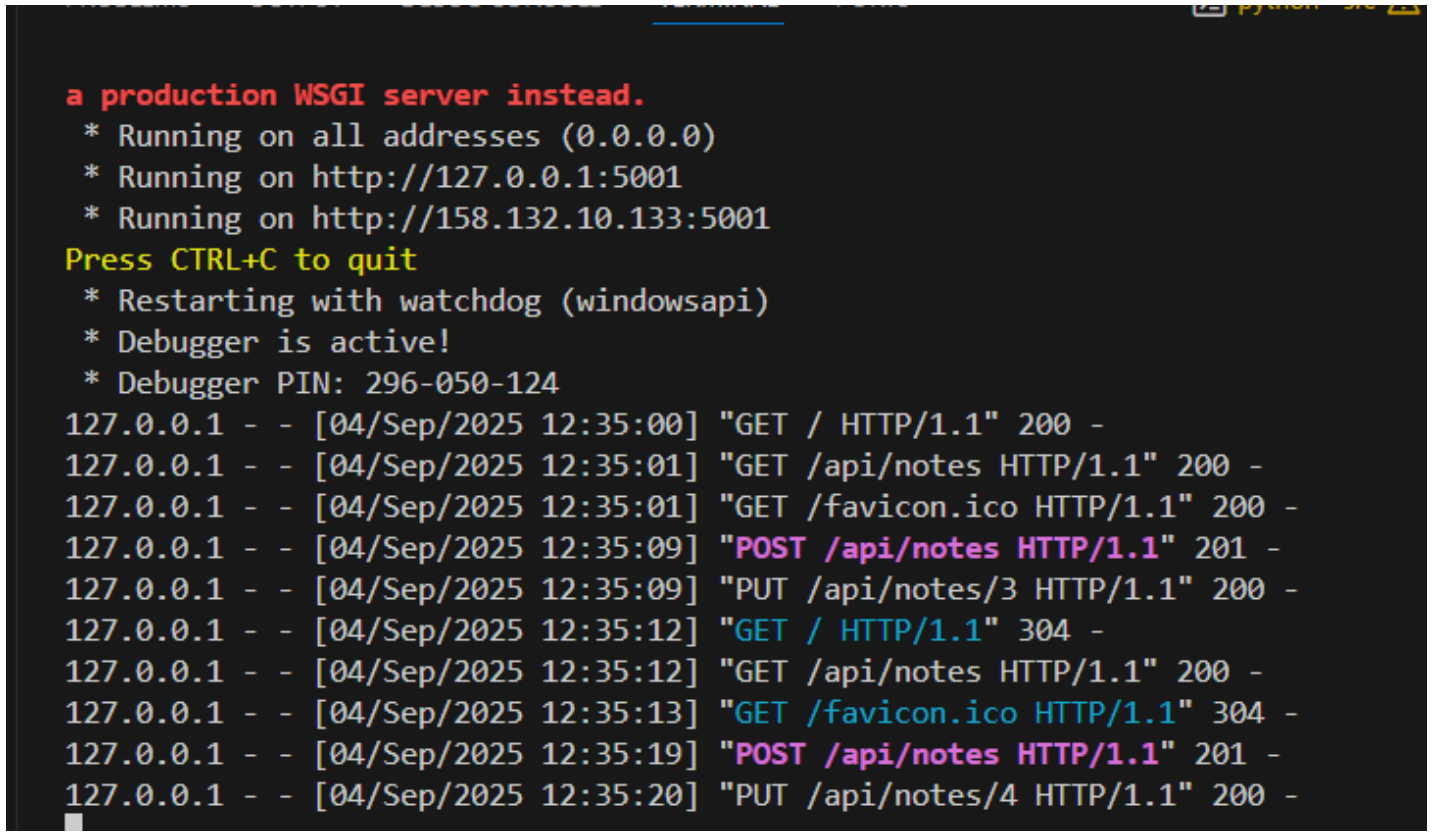
```
python3 -m venv venv
source venv/bin/activate

pip install -r requirements.txt
python src/main.py
```

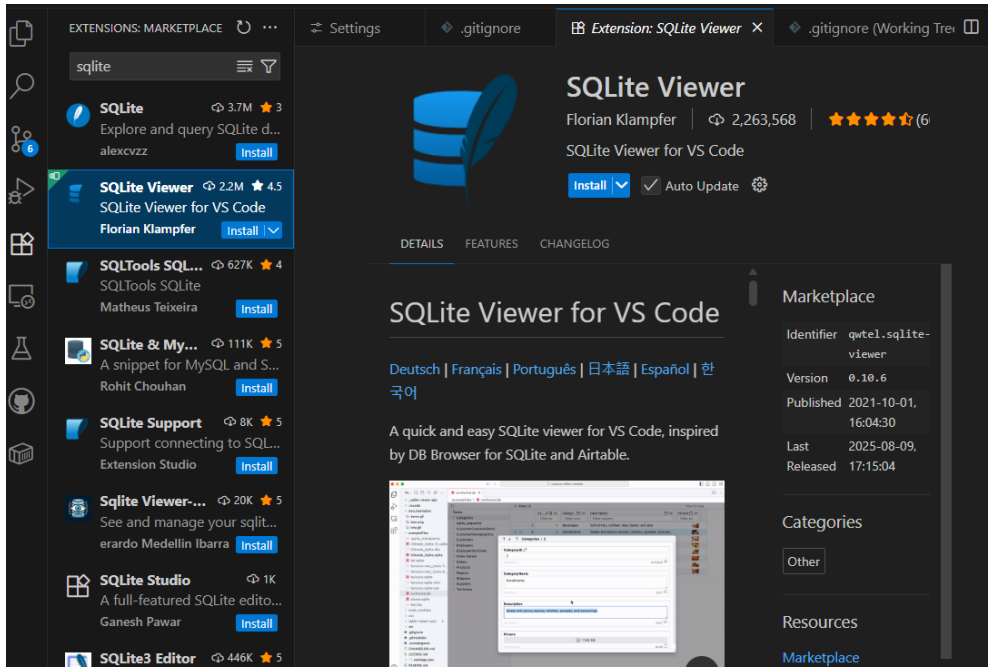
- Test the app using the exposed URL by codespaces

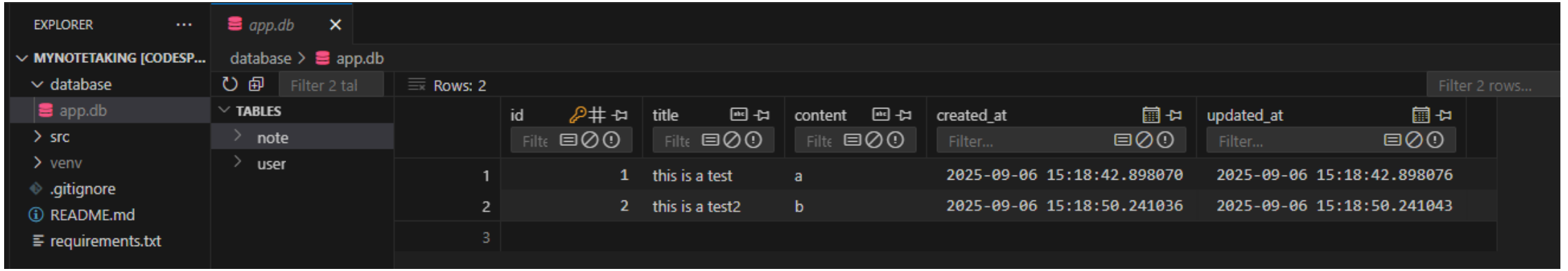


- Test the app and observe the backend logs



- Install “SQLite Viewer” VSCode plugin
- View the database





Test the app and write down your test cases and the bugs/issues that you have found

Test Cases	Bugs/Issues
	horizontal scroll bar
left pannel should	

Git

- In GitHub Codespaces, any changes made within the environment will be lost once the environment is deleted because it is temporary.
- To ensure your work is saved, you should first commit your changes to the local Git repository. After committing, push these changes to the remote repository on GitHub.
- This process ensures that all your work is stored safely and can be accessed from any future environment.

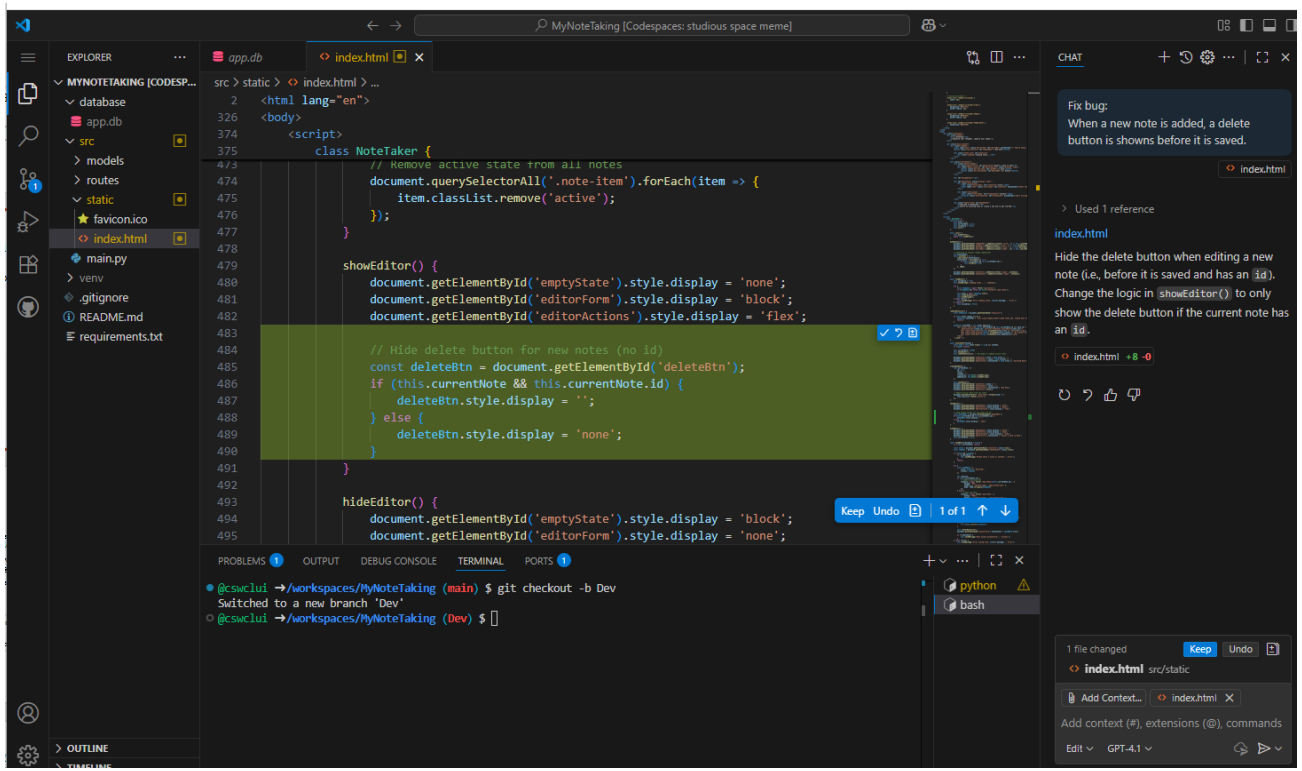
- **Git:** Version control system used to track changes in source code during software development
- **Commit:** A commit in Git is a snapshot of your changes in the repository.

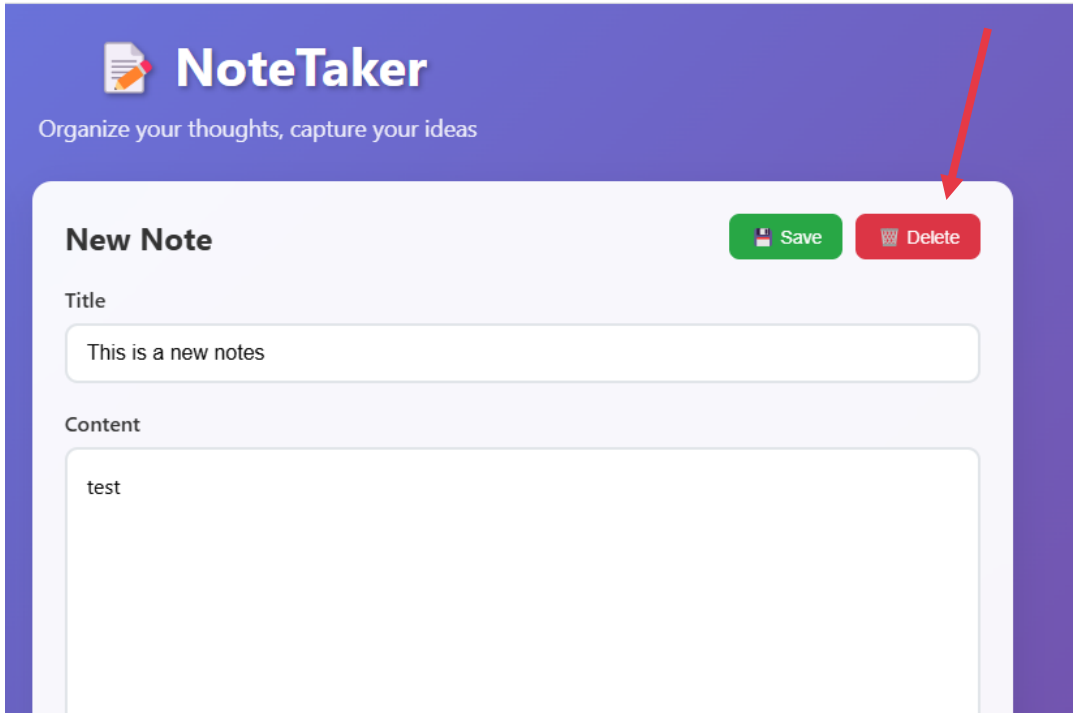
Git Branch

- **Branch**: A parallel version of the repository. It allows you to work on different features, fixes, or experiments independently from the main codebase.
- In Git, the use of branches allows you to diverge from the main line of development and continue to do work without affecting the main branch.
  - E.g., you may work on new features, bug fixes, or experiments without affecting the main codebase. Multiple developers can work on different tasks simultaneously without interfering with each other

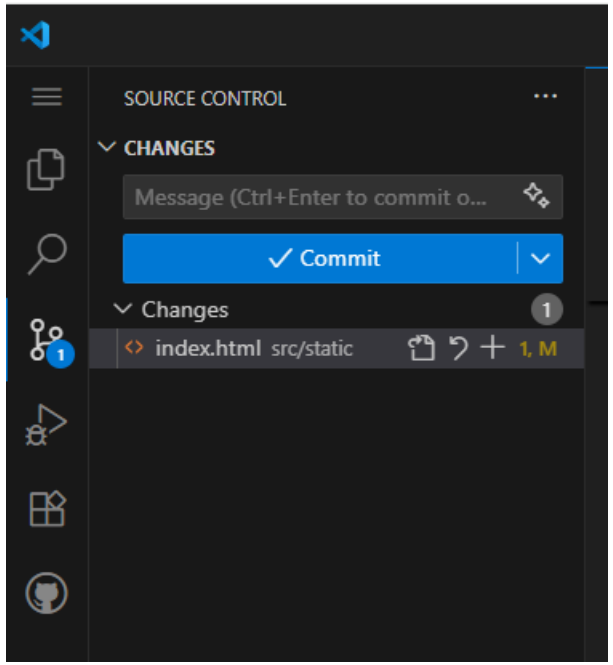
Fix a bug

- Use the coding agent to mode to fix the bug that the delete button should not appear before the notes is saved.





- Create a new branch “Dev” and checkout to the branch
- Stage and commit the change with a descriptive commit message
- Push the new commit to the GitHub repository



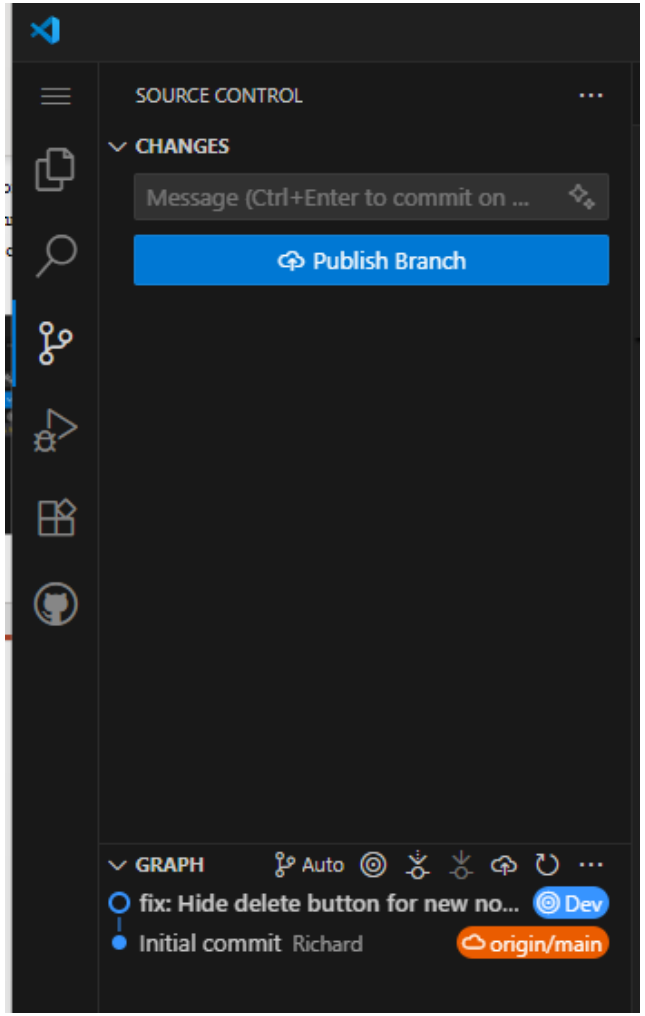

```
# Create and checkout to a new branch named "Dev"
git checkout -b Dev

# Stage your changes
git add .

# Commit with a descriptive message
git commit -m "fix: Hide delete button for new notes in the editor"

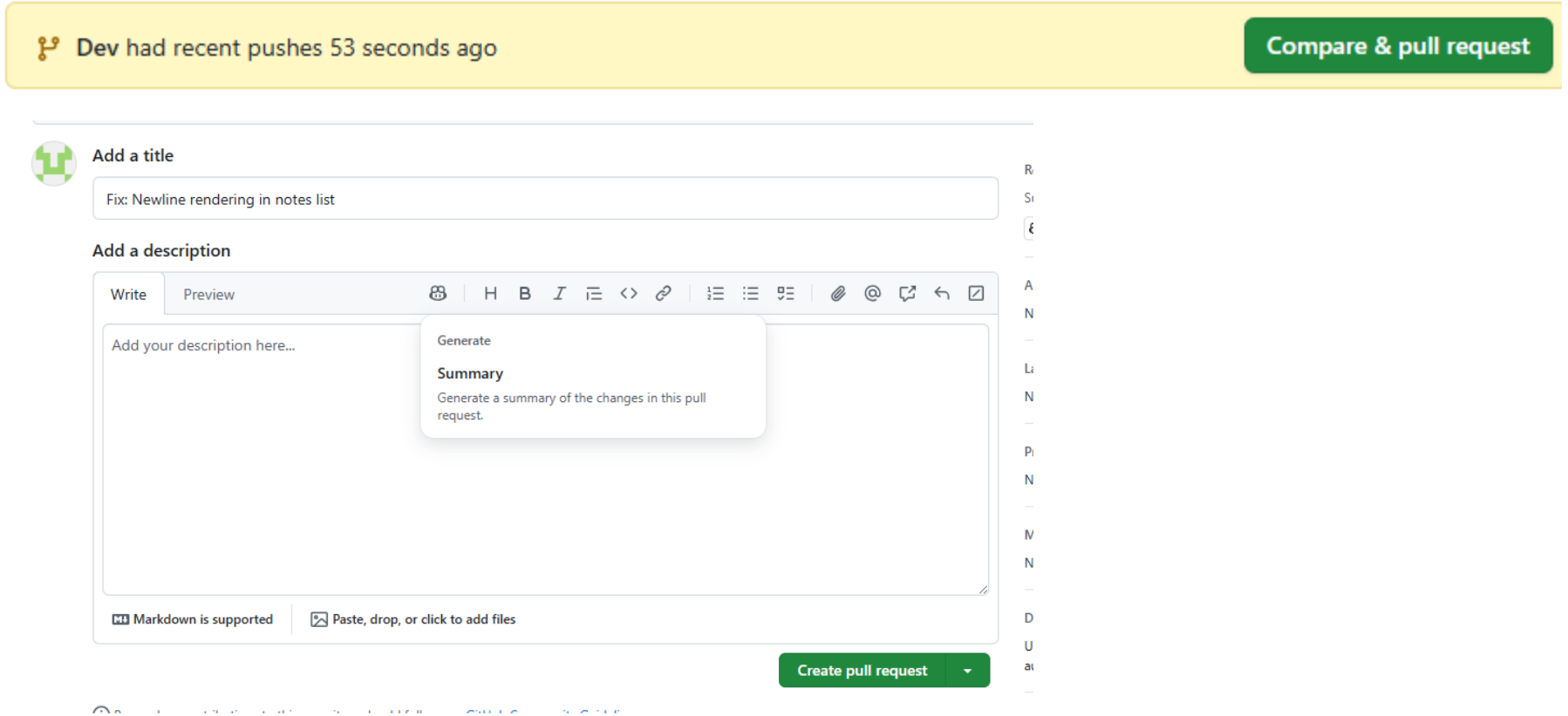
# Push the new branch and commit to GitHub
git push origin Dev
```

“feat:_____

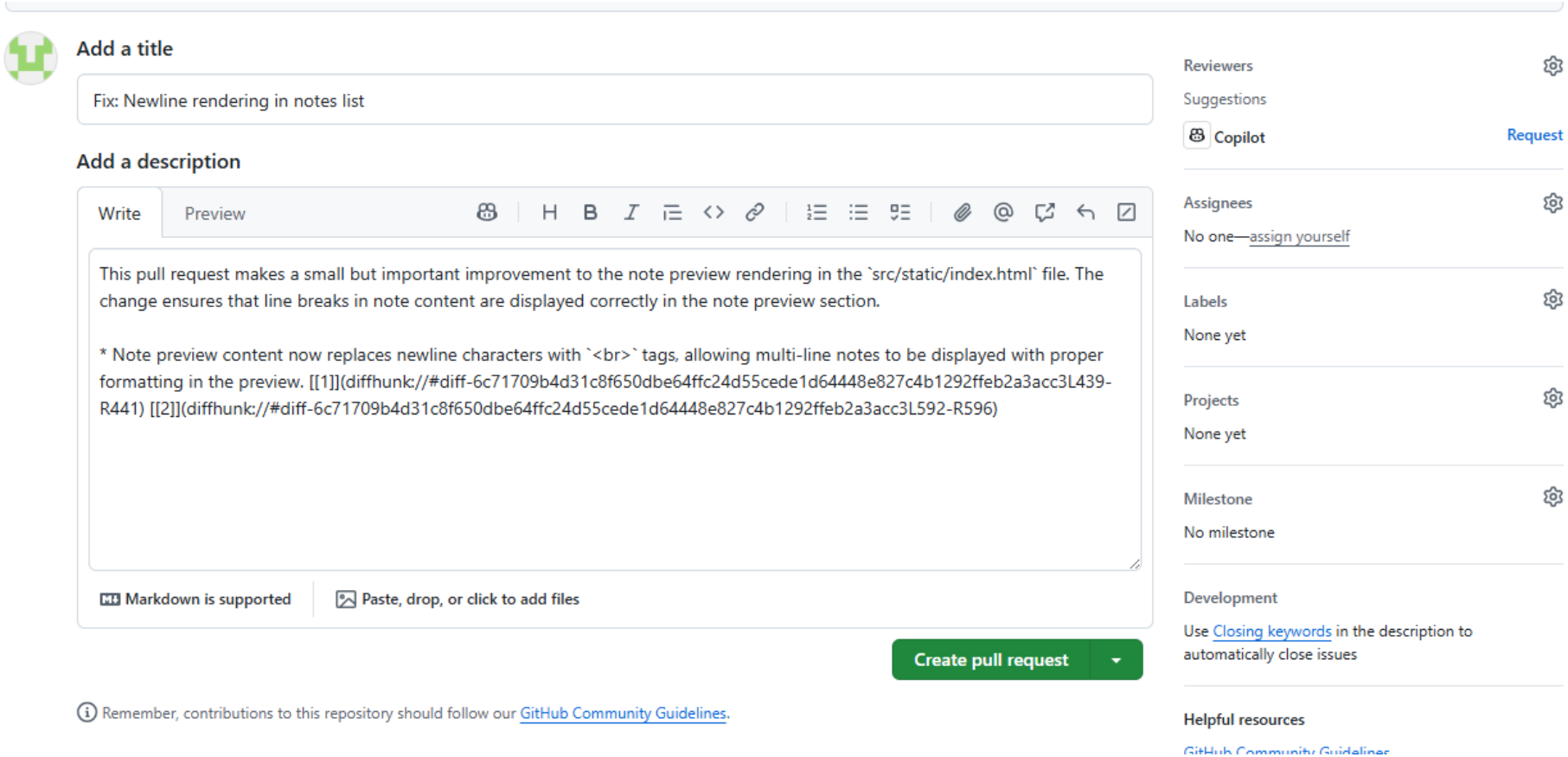



```
# Push the new branch and commit to GitHub
git push origin Dev
```

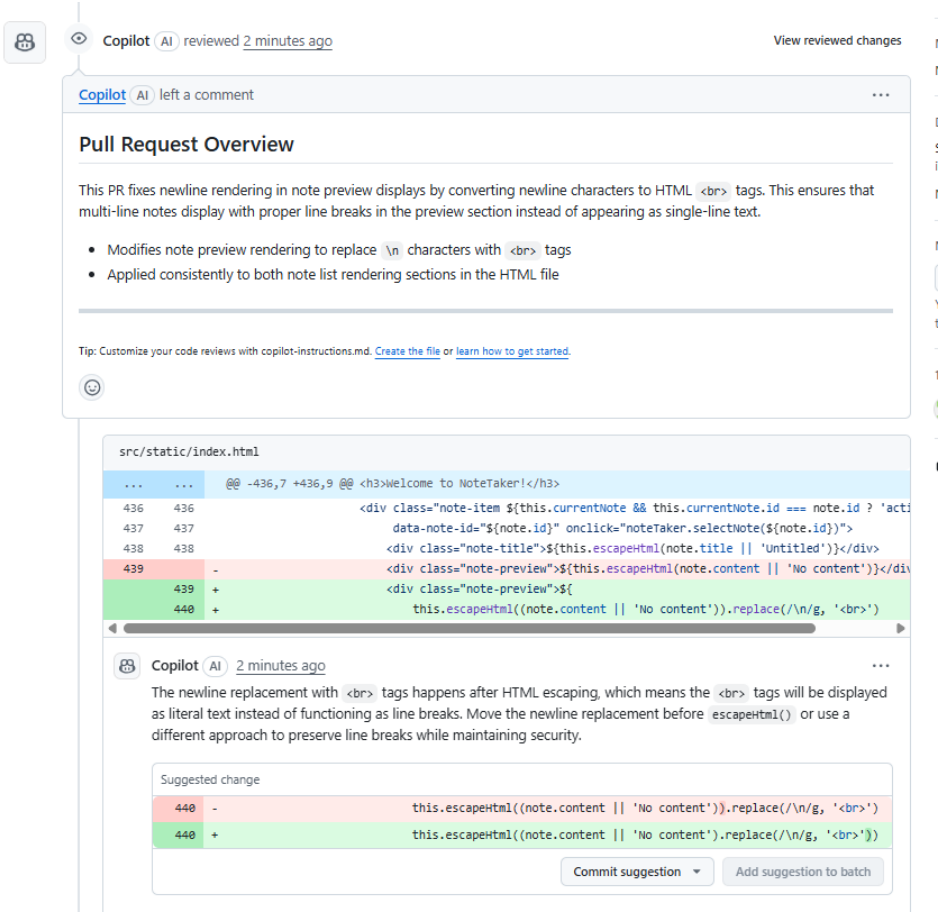
- Now, generate a pull request
  - A pull request is a formal proposal to merge code changes from one branch into another, allowing for review, discussion, and approval before integration.
- Request Copilot to generate a summary

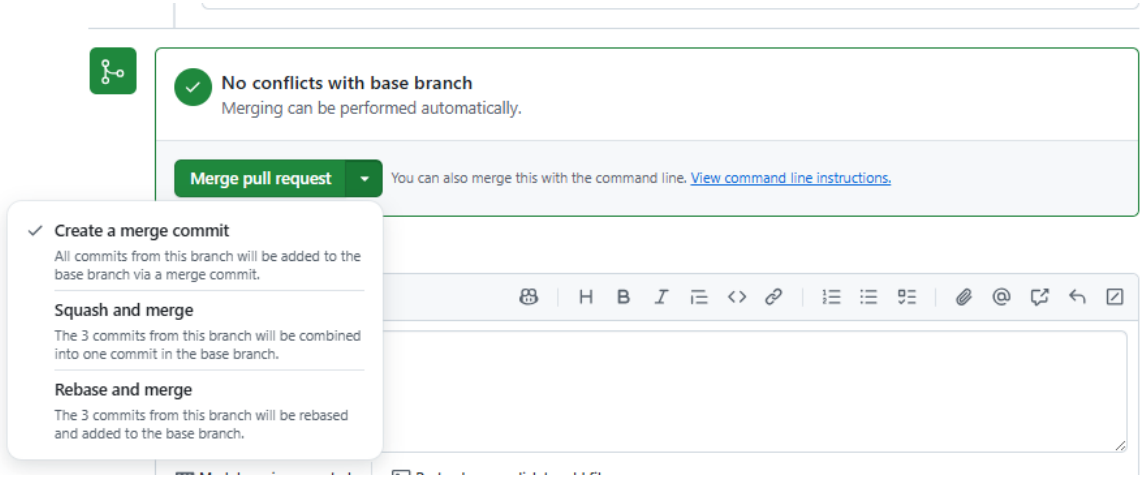


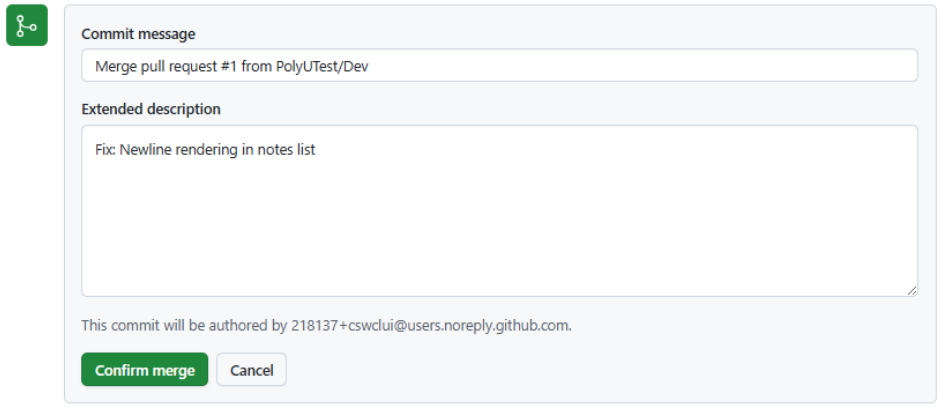
- You may request a team member/copilot to review the code change



- Review the suggestion of the Copilot coding agent.
- Merge the pull request



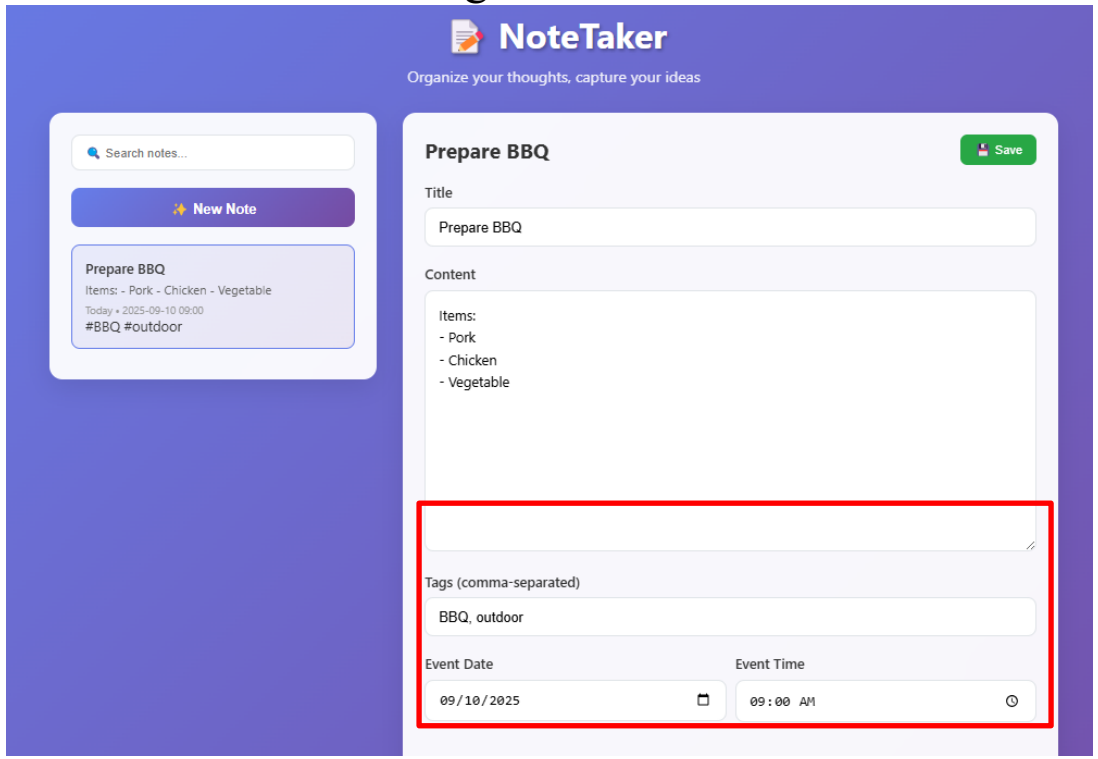


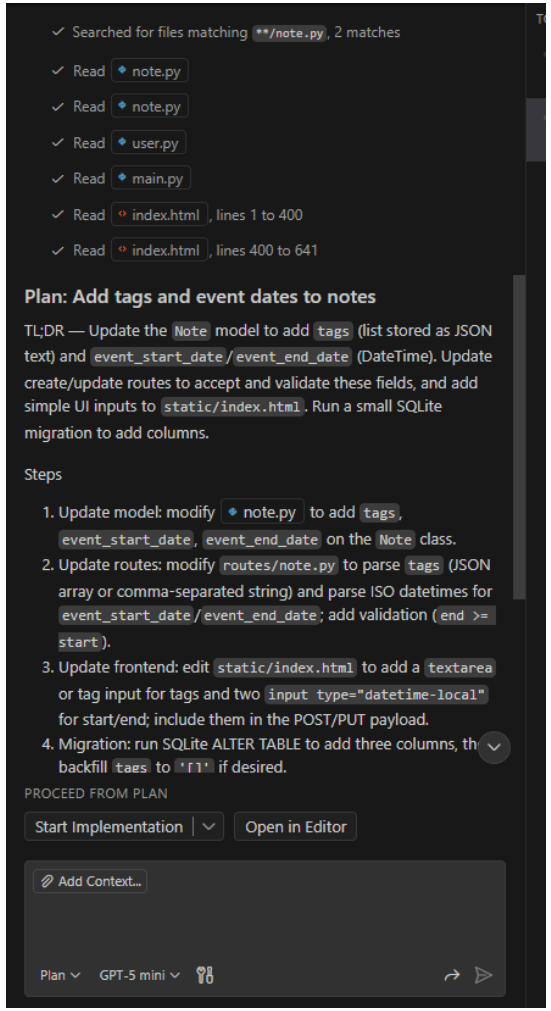


Add new fields for the notes item

- First, use the "Plan" mode to generate the plan

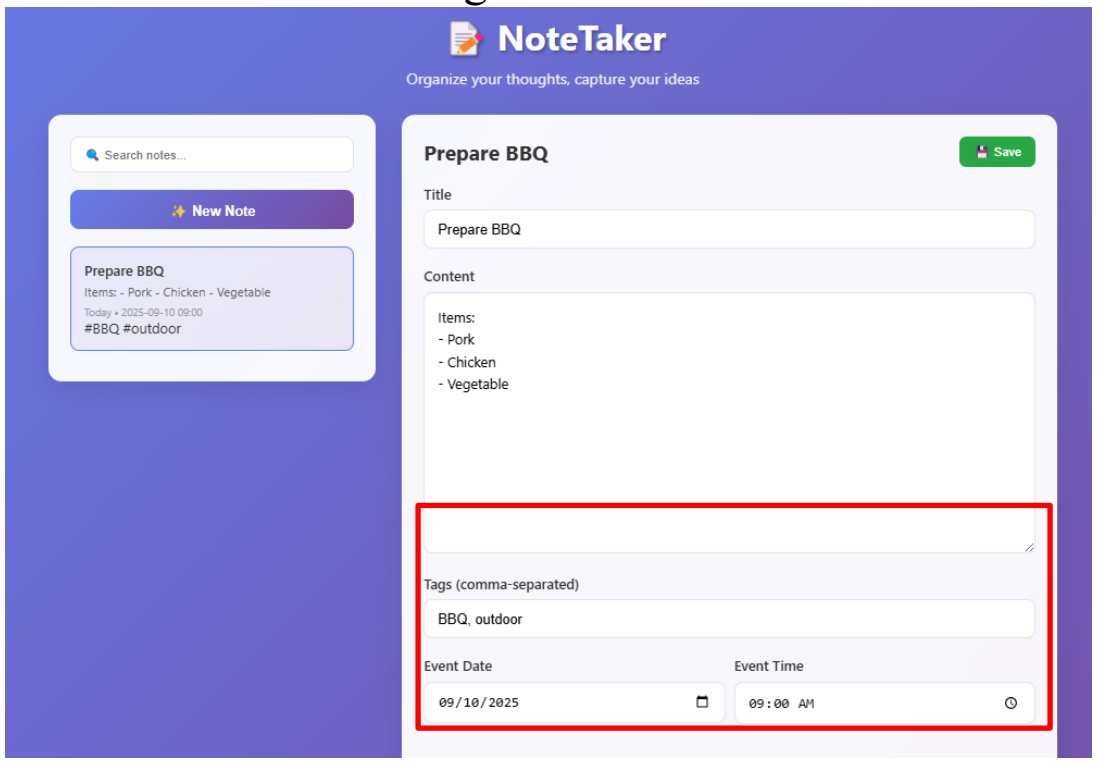
Task: add the following fields in the notes

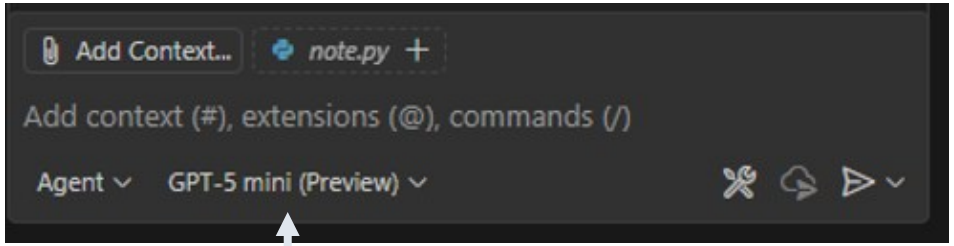




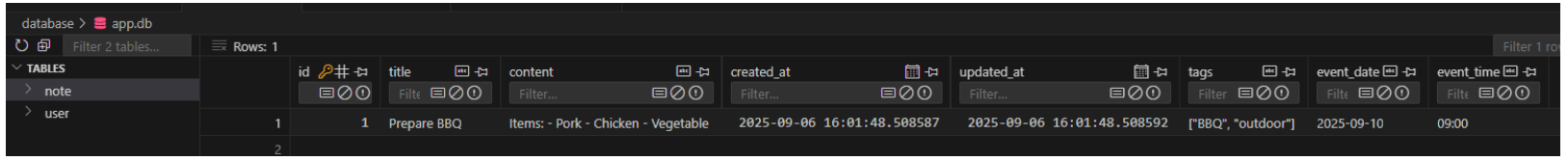
- Use the coding agent's "**Agent**" mode to implement it
- Observe how the agentic coding works.

Task: add the following fields in the notes



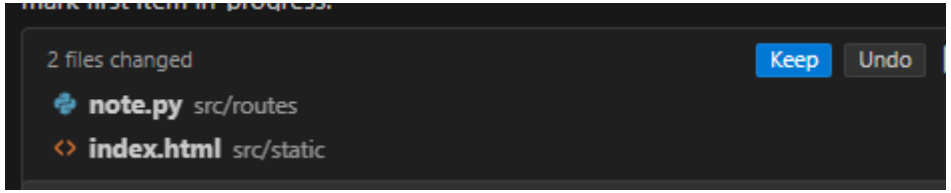


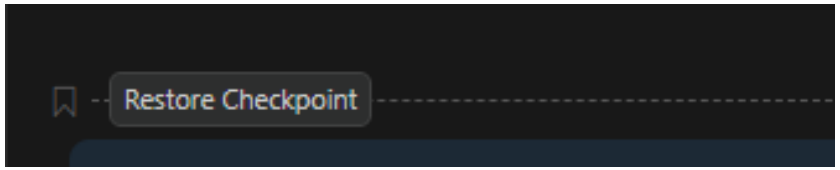
Use GPT5-mini or Claude 4.5.



Undoing the changes generated by AI

- “Undo” the AI changes
- “Restore Checkpoint” in the GitHub Copilot Chat
- Using “git reset” Command
  - Soft, Mixed vs. Hard reset

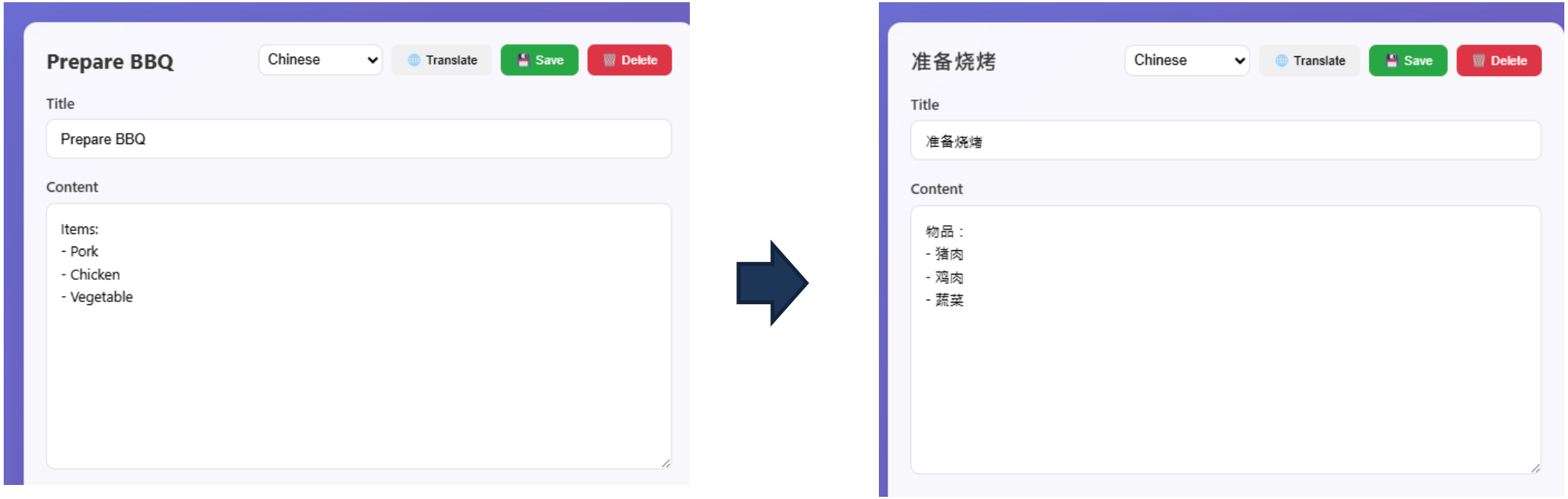




Exercise

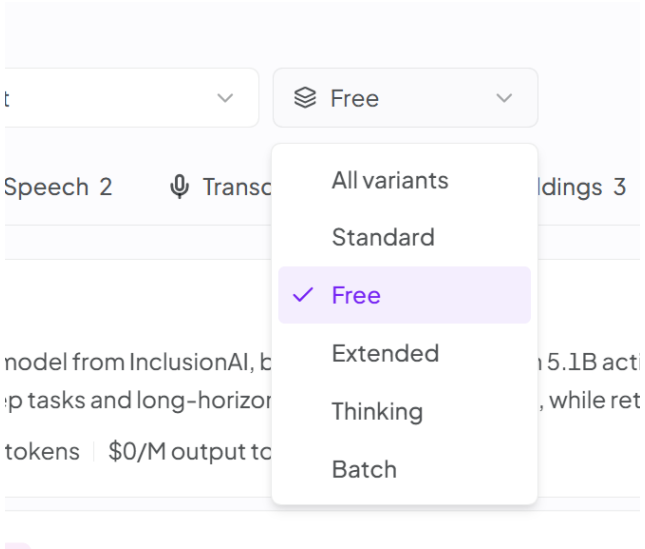
- Fix the remaining bugs of the app.
- Add new features to the app.
- Remark:
  - After the AI makes the code changes, you should test to see if the code changes is working correctly
  - Commit each change in the “Dev” branch
  - Push the commits and update the remote repository (GitHub)

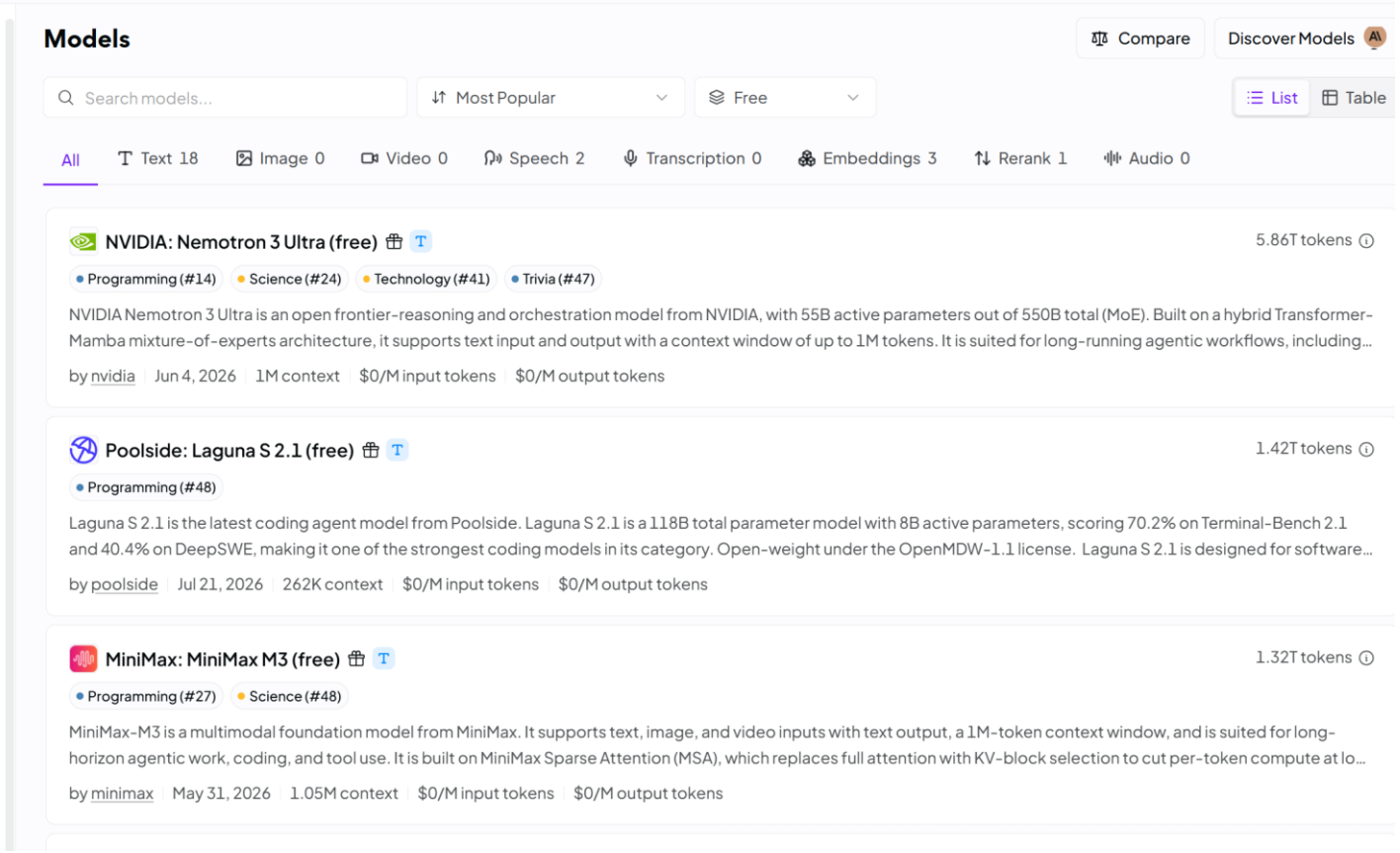
Task: Develop a translation feature for the notes



Openrouter LLM APIs

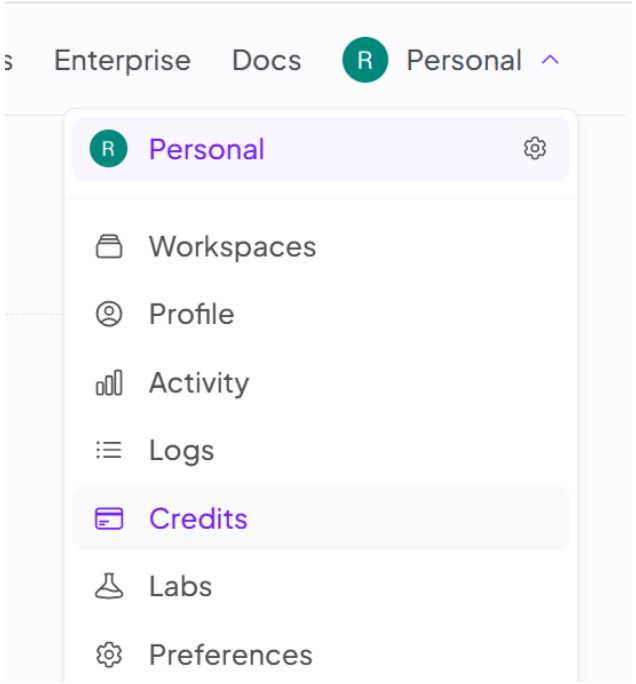
- Free models are available
- US $1 Free quota
  - Budget can get to -$1

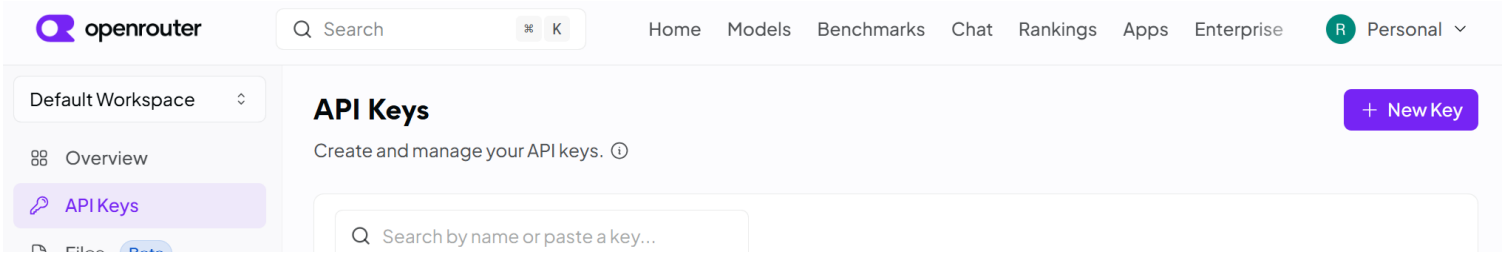




https://openrouter.ai/models

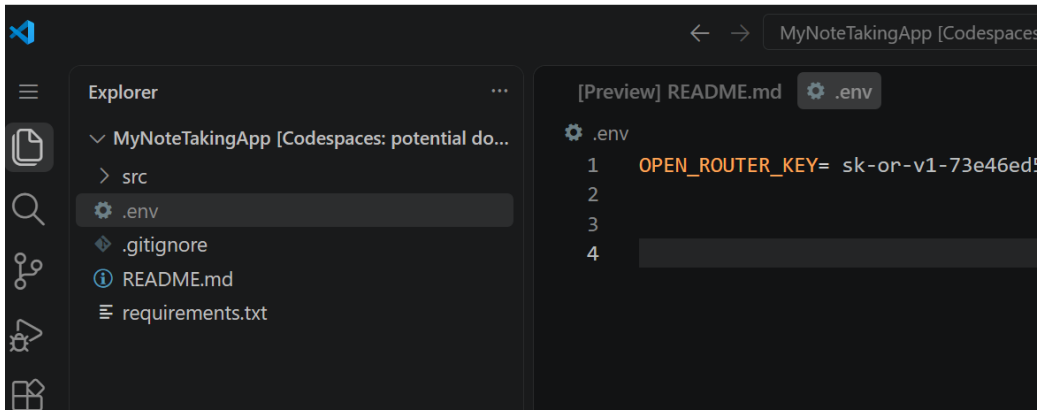
Get an API Key

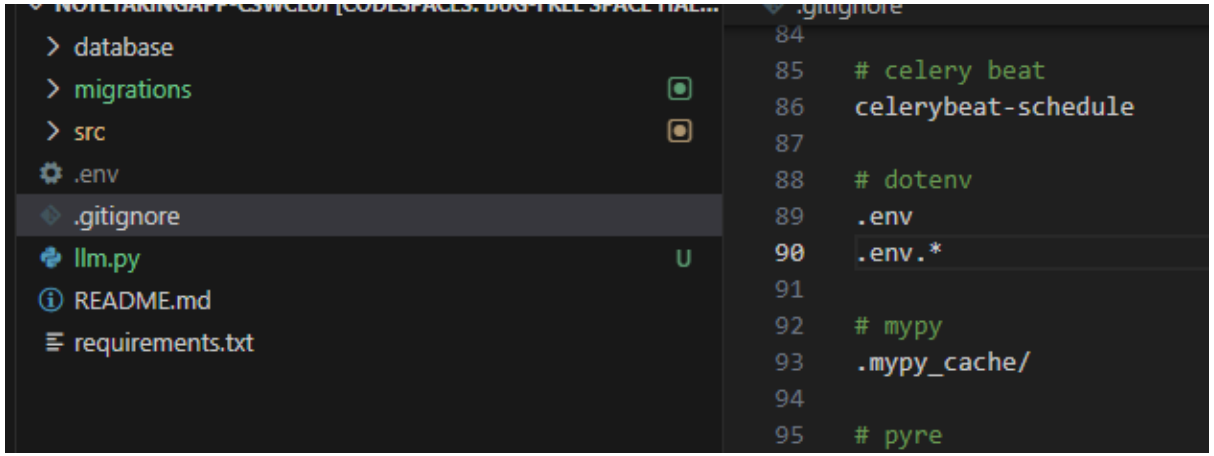




Storing API keys/credentials

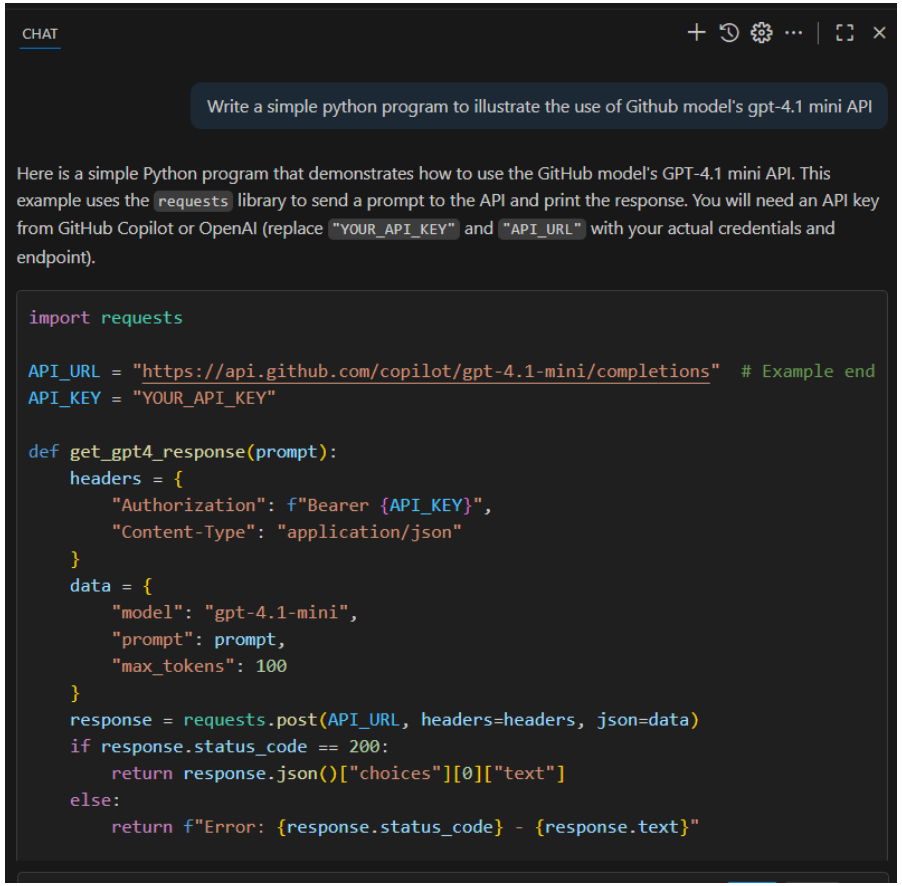
- Store the GitHub Model Credential in .env file
- Make sure the **.env** is specified **in .gitignore** file and the **.env** is not tracked by git.





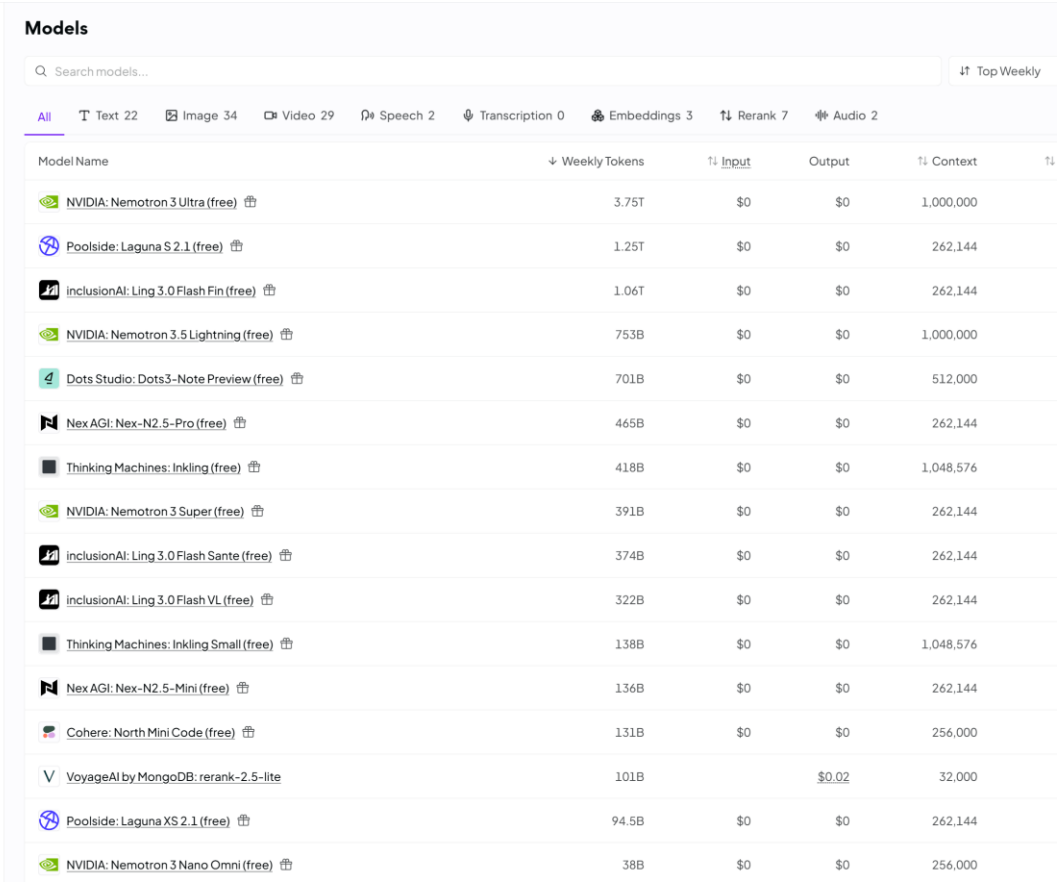
Limitations: Using LLM for coding

- Large language models (LLMs) are trained on large code corpora collected at a fixed point in time, LLMs often lack real-time updates on library evolution and API changes.
  - As a result, they may suggest code that use deprecated libraries or outdated APIs that may no longer function correctly with the latest versions of libraries or frameworks.
  - The generated code is more prone to hallucination, and might fail to compile or run, resulting in compatibility issues, bugs, or even security vulnerabilities.
- Solution:
  - Provide sample code for LLM as the context to write code



Is the code correct?

Choose a popular LLM model (cheaper/free models)



Models

Search models...

Top Weekly

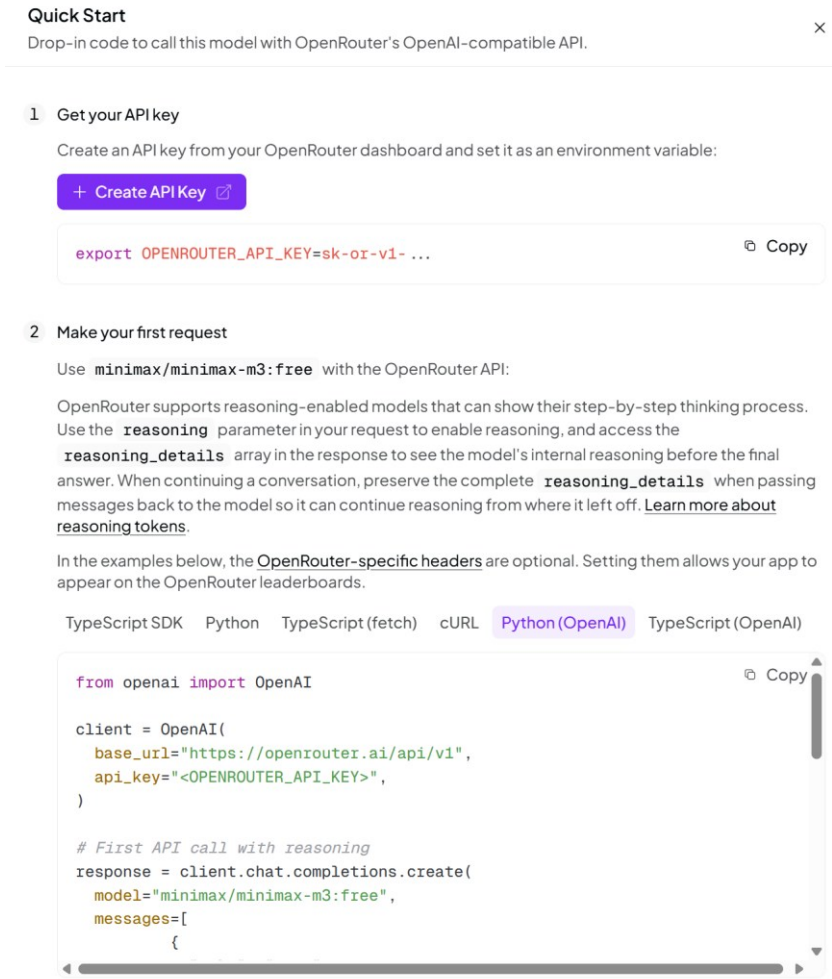
All | Text 22 | Image 34 | Video 29 | Speech 2 | Transcription 0 | Embeddings 3 | Rerank 7 | Audio 2

Model Name	Weekly Tokens	Input	Output	Context
NVIDIA: Nemotron 3 Ultra (free)	3.75T	$0	$0	1,000,000
Poolside: Laguna S 2.1 (free)	1.25T	$0	$0	262,144
inclusionAI: Ling 3.0 Flash Fin (free)	1.06T	$0	$0	262,144
NVIDIA: Nemotron 3.5 Lightning (free)	753B	$0	$0	1,000,000
Dots Studio: Dots3-Note Preview (free)	701B	$0	$0	512,000
Nex AGI: Nex-N2.5-Pro (free)	465B	$0	$0	262,144
Thinking Machines: Inkling (free)	418B	$0	$0	1,048,576
NVIDIA: Nemotron 3 Super (free)	391B	$0	$0	262,144
inclusionAI: Ling 3.0 Flash Sante (free)	374B	$0	$0	262,144
inclusionAI: Ling 3.0 Flash VL (free)	322B	$0	$0	262,144
Thinking Machines: Inkling Small (free)	138B	$0	$0	1,048,576
Nex AGI: Nex-N2.5-Mini (free)	136B	$0	$0	262,144
Cohere: North Mini Code (free)	131B	$0	$0	256,000
VoyageAI by MongoDB: rerank-2.5-lite	101B		$0.02	32,000
Poolside: Laguna XS 2.1 (free)	94.5B	$0	$0	262,144
NVIDIA: Nemotron 3 Nano Omni (free)	38B	$0	$0	256,000

https://openrouter.ai/models?max_price=0&input_modalities=text&order=top-weekly

For cheaper model, you can select use:
https://openrouter.ai/deepseek/deepseek-v4-flash-0731

- Study the sample code for Python (OpenAI)



Quick Start

Drop-in code to call this model with OpenRouter's OpenAI-compatible API.

1 Get your API key

Create an API key from your OpenRouter dashboard and set it as an environment variable:

+ Create API Key

```
export OPENROUTER_API_KEY=sk-or-v1- ...
```

Copy

2 Make your first request

Use `minimax/minimax-m3:free` with the OpenRouter API:

OpenRouter supports reasoning-enabled models that can show their step-by-step thinking process. Use the `reasoning` parameter in your request to enable reasoning, and access the `reasoning_details` array in the response to see the model's internal reasoning before the final answer. When continuing a conversation, preserve the complete `reasoning_details` when passing messages back to the model so it can continue reasoning from where it left off. Learn more about reasoning tokens.

In the examples below, the OpenRouter-specific headers are optional. Setting them allows your app to appear on the OpenRouter leaderboards.

TypeScript SDK | Python | TypeScript (fetch) | cURL | Python (OpenAI) | TypeScript (OpenAI)

Copy

```
from openai import OpenAI

client = OpenAI(
  base_url="https://openrouter.ai/api/v1",
  api_key="<OPENROUTER_API_KEY>",
)

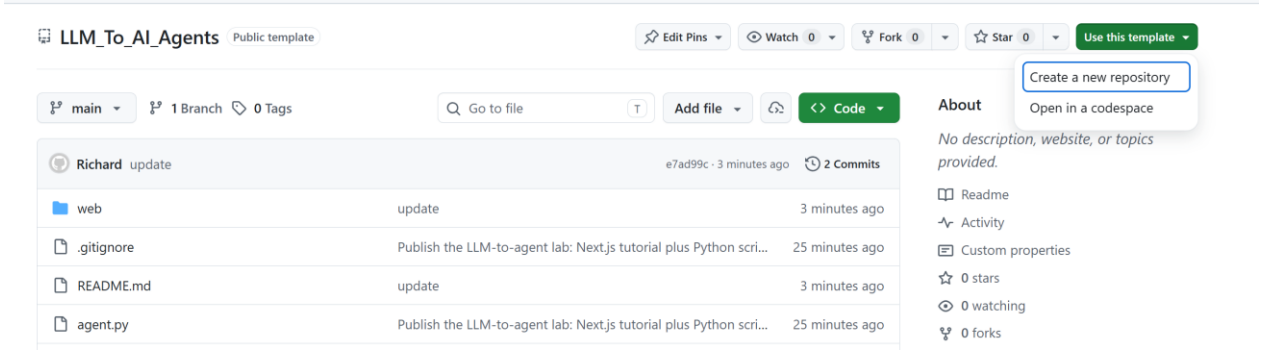
# First API call with reasoning
response = client.chat.completions.create(
  model="minimax/minimax-m3:free",
  messages=[
        {
```

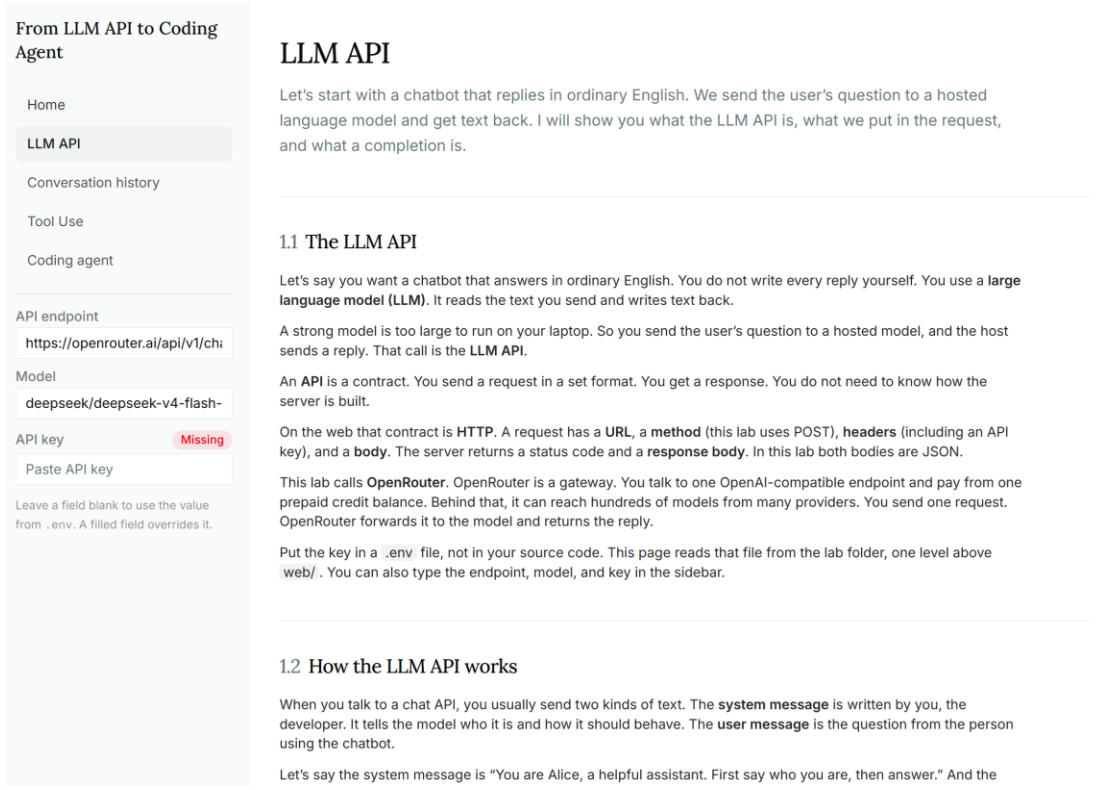
Learn about the LLM API

- Visit https://github.com/HKPolyUSE/LLM_To_AI_Agents
- Create a new repository using the template repository and start codespaces to launch the nextjs app

Run in terminal:

cd web
npm install
npm run dev





Use the coding agent to write a program using the sample code

Use the following sample code to write a python program translator.py.

read the API key from .env.

The program should define a function llm_generate() which accepts a user prompt read from the command line.

It should define a system prompt to act as a professional translator to translate the user input into the target language

Target language: **{put your target language here}**

Put your sample program below

{put the sample code from Openrouter here …}

Run the program

- Add the library in requirements.txt
  - openai==1.106.1
  - dotenv==0.9.9
- Install the library in the virtual environment
  - pip install -r requirements.txt
- Run the app
  - python translator.py “How are you?”

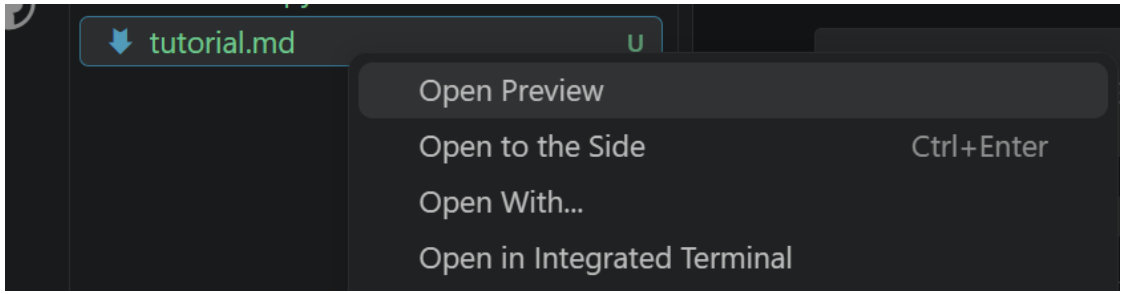


Use AI to create a tutorial to explain the program

- Exercise: Use the following prompt to ask AI to explain how translator.py works.

  Write a tutorial to explain how this program works. output in tutorial.md

- Preview the markdown file in VSCode (right click the file->Open Preview)



Step-by-step flow

a) Get the API key

```
api_key = _resolve_api_key()
```

b) Create the OpenAI client

```
client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=api_key,
)
```

This tells the Python OpenAI SDK to send requests to OpenRouter instead of OpenAI's default API.

c) Build a system prompt

```
system_prompt = (
    f"You are a professional translator. Translate the user's text into {TARGET_LANGUAGE}. "
    "Preserve the original meaning, tone, and intent. Return only the translated text."
)
```

This prompt tells the LLM:

- be a professional translator
- translate into the target language
- keep the meaning and tone
- return only the translation

d) Send the request to the model

```
response = client.chat.completions.create(
    model="minimax/minimax-m3:free",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ],
    temperature=0.3,
)
```

How translator.py talks to the LLM

API key → OpenRouter client → system + user prompts → chat.completions.create

1. Load the API key

.env

OPEN_ROUTER_KEY=sk-or-...

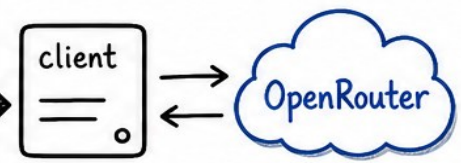
_resolve_api_key()

- Reads OPEN_ROUTER_KEY from the environment
- Returns that value

If missing, raise ValueError

2. Create the OpenAI client

```
client = OpenAI(
  base_url="https://openrouter.ai/api/v1",
  api_key=api_key
)
```



OpenAI SDK, but endpoint is OpenRouter, not openai.com

api_key is sent as the Authorization header on every request.

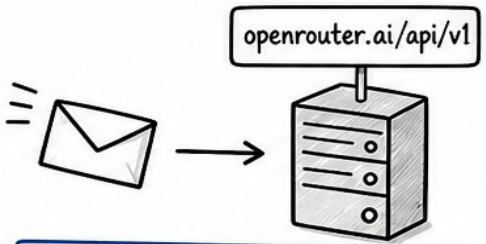
3. Build the two prompts

```
messages = [
```

1. role: system — "You are a professional translator. Translate the user's text into Chinese. Preserve meaning, tone, and intent. Return only the translated text."
2. role: user — user_prompt (the text typed on the command line)

```
$ python translator.py
"Hello world"
```

4. Send to the API endpoint



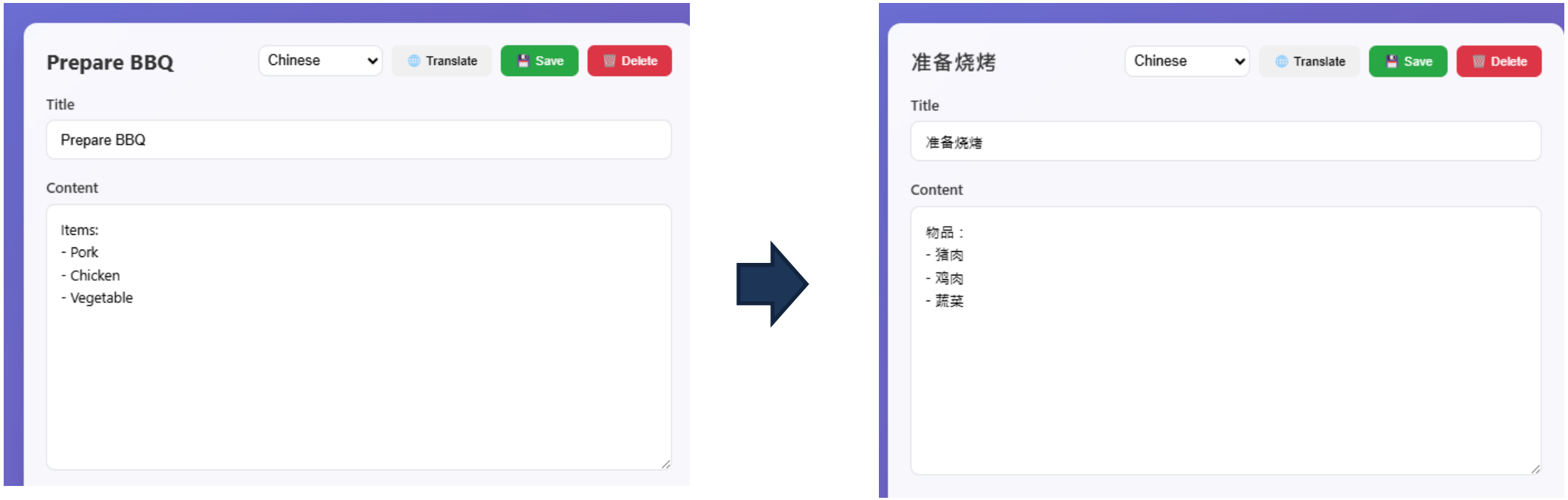

```
POST openrouter.ai/api/v1
model = "minimax/minimax-m3:free"
messages = messages
temperature = 0.3
```

Reply: response.choices[0].message.content

If auth fails → AuthenticationError → tell user the key is invalid.

Develop a new feature to translate the notes using LLM APIs

1. Start a new session in the coding agent. Add **translator.py** as the context of the prompt for when writing the code
2. Prompt the coding agent (with translator.py as the context):
   Create the translation feature in the note taking app. Store the prompt file in the folder prompts/translate_prompt.md



Use AI to explain what code changes are made when creating the feature.

Commit change in Dev branch

- Use the GitHub Copilot Agent mode to add the translation function to the App
- Stage the changes
- Commit with message **"feat: Translate notes"**

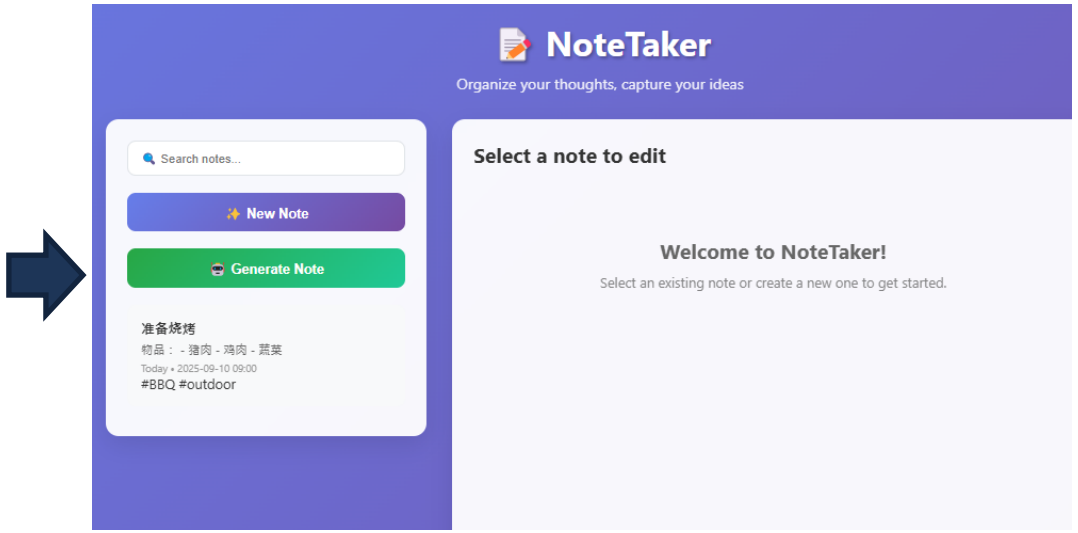
```
# Create and checkout to a new branch named "Dev“
# If you already have the branch created before, use “git checkout Dev” to change to the branch

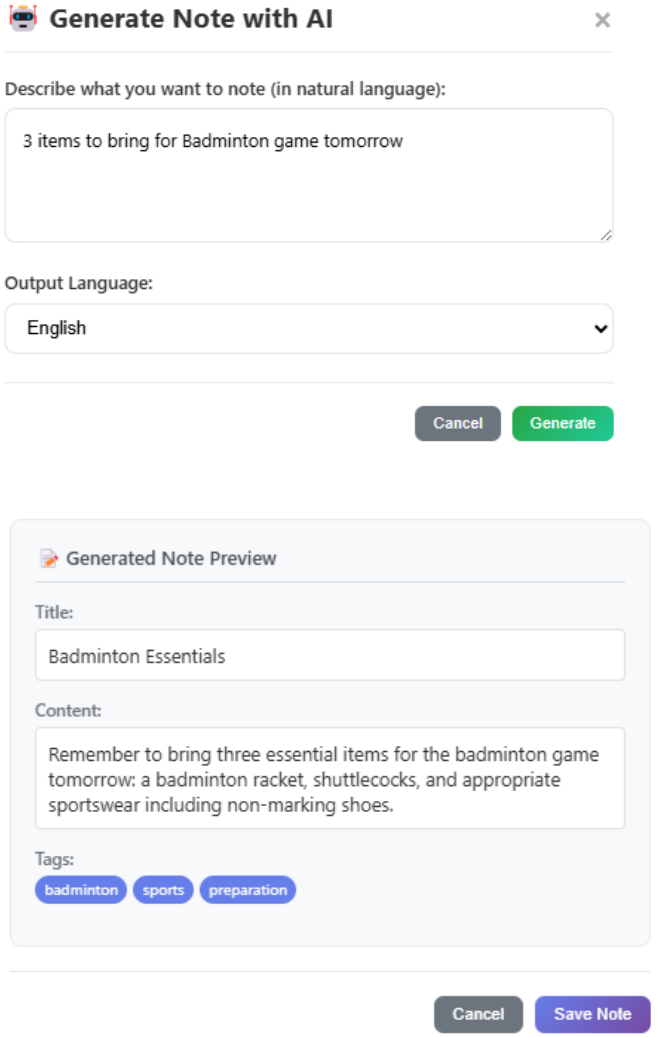
git checkout -b Dev

# Stage your changes and commit
git add .
git commit -m "feat: Translate notes"

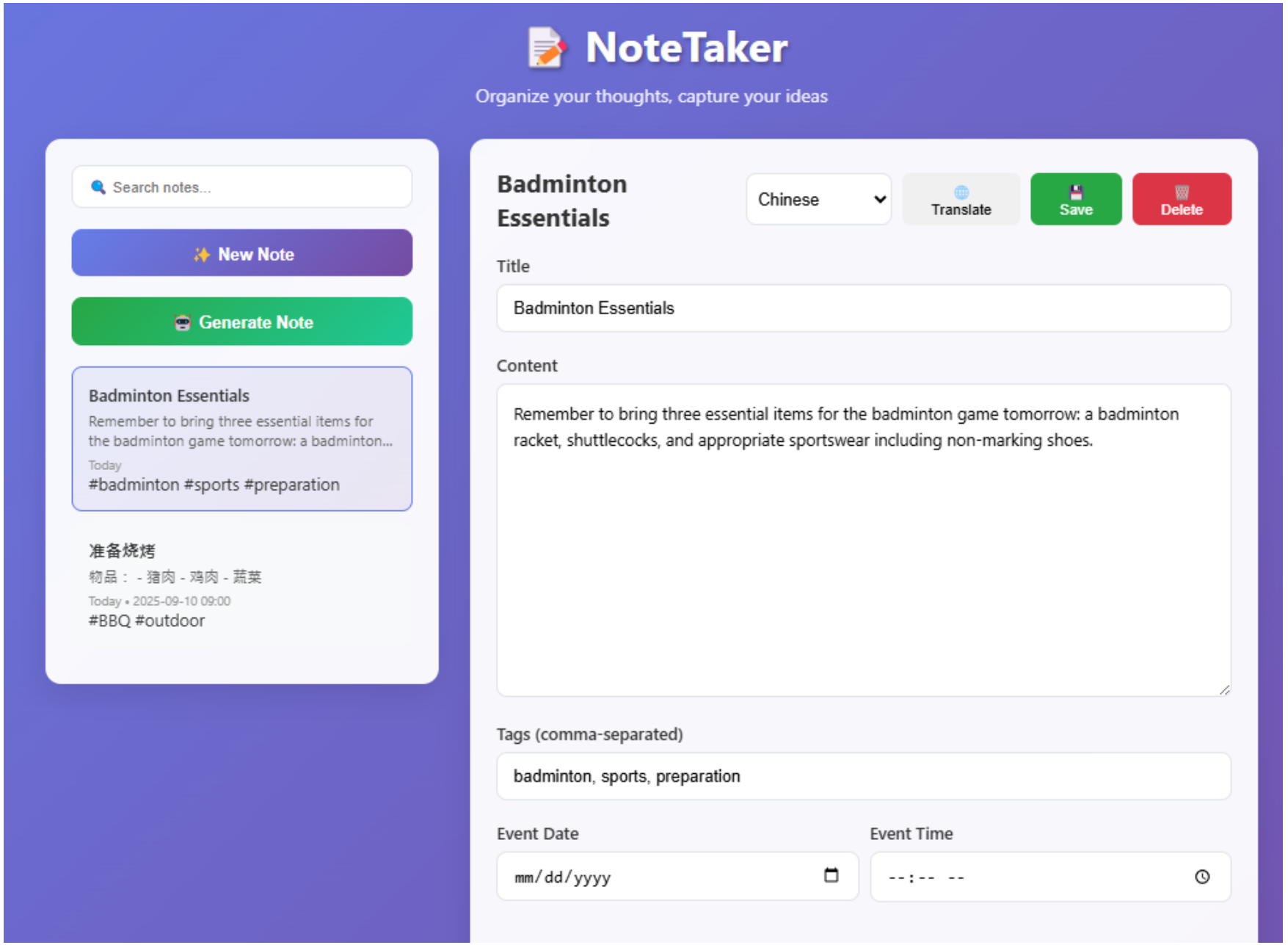
# Push the new branch and commit to GitHub
git push origin Dev
```

Add feature: “Generate Notes”





Allow the user to input the notes in natural language.
LLM is used to generate the title, contents of the notes with three suggested tags



The AI generated notes with title, content, and tags

Add these fields to your app and DB first

- First, we implement the helper function to generate the notes in structured JSON format.

```
system_prompt = '''
Extract the user's notes into the following structured fields:
1. Title: A concise title of the notes less than 5 words
2. Notes: The notes based on user input written in full sentences.
3. Tags (A list):  At most 3 Keywords or tags that categorize the content of the notes.


Output in JSON format without ```json. Output title and notes in the language: {lang}.
Example:
Input: "Badminton tmr 5pm @polyu".
Output:
{{
  "Title": "Badminton at PolyU",
  "Notes": "Remember to play badminton at 5pm tomorrow at PolyU.",
  "Tags": ["badminton", "sports"]
}}

'''
```

Alternatively, you may also store the system prompt in a file (to facilitating easy customization).

```python
def process_user_notes(language, user_input):
    system_prompt_filled = system_prompt.format(
        lang=language
    )

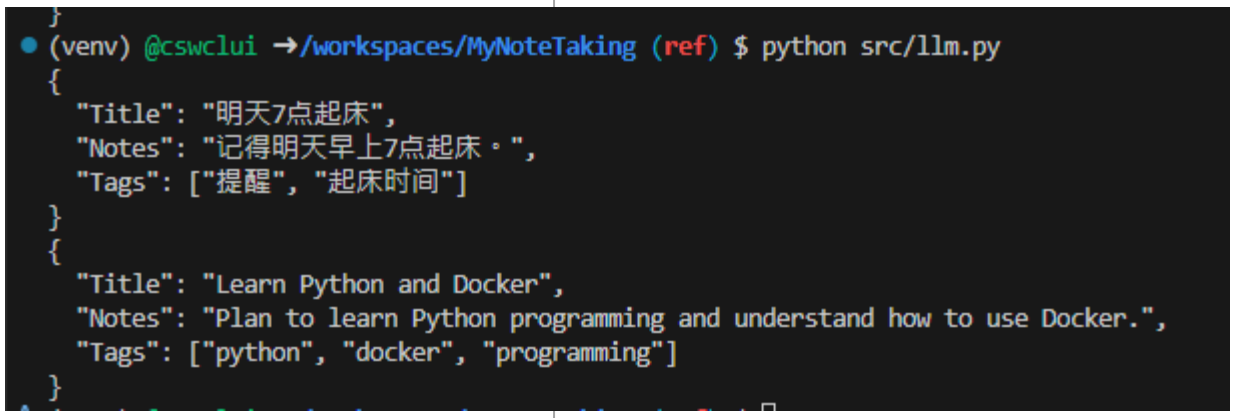
    messages = [
        {
            "role": "system",
            "content": system_prompt_filled,
        },
        {
            "role": "user",
            "content": user_input,
        }
    ]

    response_content = call_llm_model(model, messages)
    return response_content

# Run the main function if this script is executed
if __name__ == "__main__":
   result = process_user_notes("Chinese", "Get up tomorrow 7am")
   print(result)

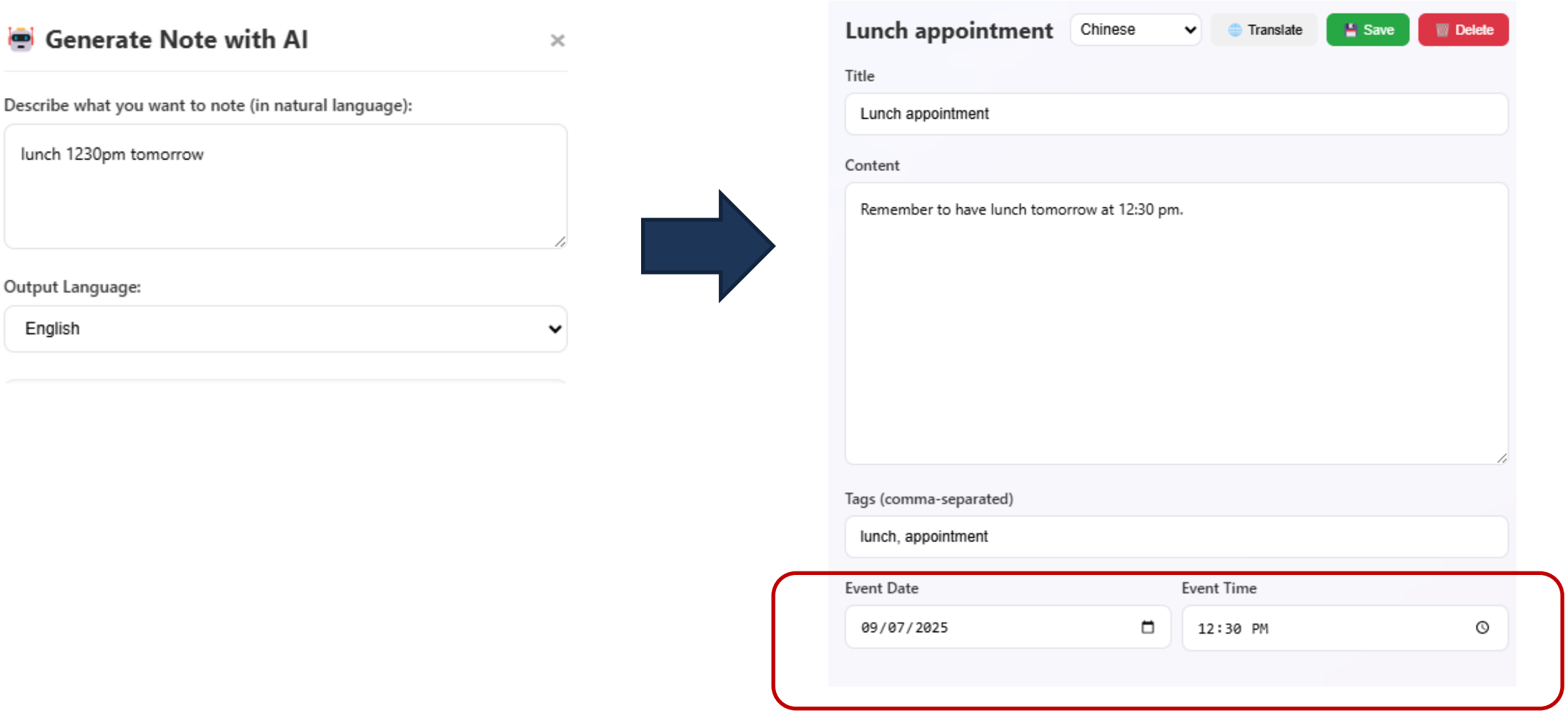
   result = process_user_notes("English", "Learn python programming and docker")
   print(result)
```

Run the program



Use the agent mode and ask AI to implement the feature by using the above helper function.

Exercise: Extend the “Generate notes” function to extract the date and time (if available)



You can first modify the helper function to extract the data and time from user input.
Then, use it as the context for the Copilot Agent to modify the app.

```
result = process_user_notes("Chinese", "Get up tomorrow 7am")
print(result)
```

```
(venv) @cswclui ➜/workspaces/MyNoteTaking (ref) $ python src/llm.py
{
  "Title": "明天早上7点起床",
  "Notes": "提醒自己明天早上7点起床。",
  "Tags": ["起床", "提醒"],
  "Date": "2025-09-07",
  "Time": "07:00"
}
```

LLM doesn’t have any knowledge about the current date/time.
How can we let the LLM knows the date for tomorrow?

Copilot instruction

- Defined in a markdown file named **.github/copilot-instructions.md**
  - Natural language guidance to help Github Copilot generate code and responses better tailored to a project's specific needs, team workflows, coding standards, and preferences.
  - E.g. You may define your coding style preference, how you want APIs called, naming patterns you want followed, etc
- The Copilot Agent will first read the file to understand the project contexts/rules before completing the actual task
- Purpose
  - Help align AI-generated code with human expectations, project standards, and team workflows.
  - Help the AI understand project-specific context, business logic, and domain knowledge

Example 1

```
This is a Next.js-based travel application with TypeScript that helps users search for
trips, manage bookings, view travel guides, and track points. The application uses
React components, server components, and client components as part of the Next.js App
Router architecture. Please follow these guidelines when contributing:

## Code Standards

### Required Before Each Commit
- Run `npm run lint` to ensure code follows project standards
- Make sure all components follow Next.js App Router patterns
- Client components should be marked with 'use client' when they use browser APIs or
React hooks
- When adding new functionality, make sure you update the README
- Make sure that the repository structure documentation is correct and accurate in the
Copilot Instructions file
- Ensure all tests pass by running `npm run test` in the terminal

### TypeScript and React Patterns
- Use TypeScript interfaces/types for all props and data structures
- Follow React best practices (hooks, functional components)
- Use proper state management techniques
- Components should be modular and follow single-responsibility principle

### Styling
- You must prioritize using Tailwind CSS classes as much as possible. If needed, you
may define custom Tailwind Classes / Styles. Creating custom CSS should be the last
approach.

## Development Flow
- Install dependencies: `npm install`
- Development server: `npm run dev`
- Build: `npm run build`
- Test: `npm run test`
- Lint: `npm run lint`
```

```
## Development Flow
- Install dependencies: `npm install`
- Development server: `npm run dev`
- Build: `npm run build`
- Test: `npm run test`
- Lint: `npm run lint`

## Repository Structure
- `app/`: Next.js App Router pages and layouts organized by route
- `components/`: Reusable React components
  - `components/ui/`: UI components (buttons, inputs, etc.)
  - `components/__tests__/`: Component tests
- `lib/`: Core logic and services
  - `lib/data/`: Data models and mock data
  - `lib/types/`: TypeScript type definitions
- `public/`: Static assets
- `tests/`: Test files and test utilities
- `README.md`: Project documentation

## Key Guidelines
1. Make sure to evaluate the components you're creating, and whether they need 'use
client'
2. Images should contain meaningful alt text unless they are purely for decoration. If
they are for decoration only, a null (empty) alt text should be provided (alt="") so
that the images are ignored by the screen reader.
3. Follow Next.js best practices for data fetching, routing, and rendering
4. Use proper error handling and loading states
5. Optimize components and pages for performance
```

https://github.blog/ai-and-ml/github-copilot/copilot-ask-edit-and-agent-modes-what-they-do-and-when-to-use-them

Example 2

Project Awareness & Context

- **Always read** `PLANNING.md` at the start of a new conversation to understand the project's architecture, goals, style, and constraints.
- **Check** `TASK.md` before starting a new task. If the task isn't listed, add it with a brief description and today's date.
- **Use consistent naming conventions, file structure, and architecture patterns** as described in `PLANNING.md`.
- **Use venv_linux** (the virtual environment) whenever executing Python commands, including for unit tests.

Code Structure & Modularity

- **Never create a file longer than 500 lines of code.** If a file approaches this limit, refactor by splitting it into modules or helper files.
- **Organize code into clearly separated modules**, grouped by feature or responsibility. For agents this looks like:
  - `agent.py` - Main agent definition and execution logic
  - `tools.py` - Tool functions used by the agent
  - `prompts.py` - System prompts
- **Use clear, consistent imports** (prefer relative imports within packages).
- **Use clear, consistent imports** (prefer relative imports within packages).
- **Use python_dotenv and load_env()** for environment variables.

Testing & Reliability

- **Always create Pytest unit tests for new features** (functions, classes, routes, etc).
- **After updating any logic**, check whether existing unit tests need to be updated. If so, do it.
- **Tests should live in a** `/tests` **folder** mirroring the main app structure.
  - Include at least:
    - 1 test for expected use
    - 1 edge case
    - 1 failure case

Task Completion

- **Mark completed tasks in** `TASK.md` immediately after finishing them.
- Add new sub-tasks or TODOs discovered during development to `TASK.md` under a "Discovered During Work" section.

Style & Conventions

- **Use Python** as the primary language.
- **Follow PEP8**, use type hints, and format with `black`.
- **Use** `pydantic` **for data validation.**
- Use `FastAPI` for APIs and `SQLAlchemy` or `SQLModel` for ORM if applicable.
- Write **docstrings for every function** using the Google style:

```
def example():
    """
    Brief summary.

    Args:
        param1 (type): Description.

    Returns:
        type: Description.
    """
```

Documentation & Explainability

- **Update** `README.md` when new features are added, dependencies change, or setup steps are modified.
- **Comment non-obvious code** and ensure everything is understandable to a mid-level developer.
- When writing complex logic, **add an inline** `# Reason:` **comment** explaining the why, not just the what.

AI Behavior Rules

- **Never assume missing context. Ask questions if uncertain.**
- **Never hallucinate libraries or functions** – only use known, verified Python packages.
- **Always confirm file paths and module names** exist before referencing them in code or tests.
- **Never delete or overwrite existing code** unless explicitly instructed to or if part of a task from `TASK.md`.

https://github.com/coleam00/context-engineering-intro/blob/main/CLAUDE.md

Exercise: Deploying your app to Vercel

- Let's deploy your notetaking app to https://vercel.com!
  - Our existing app makes use of SQLite for storing data (file-based database)
  - Vercel's serverless deployment model does not support the persistent, writable filesystem that SQLite depends on.
  - The filesystem available to serverless functions in Vercel is ephemeral (wiped on every request)
    - Each serverless function runs in a stateless and often read-only environment
    - The app cannot reliably write to or read from a local database file
- Complete the following task with coding agents.
  - Step 1: Refactor the app to store the data in an external database
    - E.g., Use a cloud database (e.g. Store in the Postgres DB in Supabase)
    - Test that the app can successfully store the data in the database
  - Step 2: Refactor the app structure such that it can be deployed and run successfully in the Vercel cloud platform.
    - The LLM API key should be retrieved from the environment variable (configured in the Vercel platform)
  - Step 3:
    - Add the feature to allow the user to attach documents (e.g. PDF) to notes. The notes should be stored in the object store in Supabase
  - Optional features:
    - Add unique features to your notetaking app to make it more useful and innovative.
    - Improve the UI/UX of the app

Resources

- https://code.claude.com/docs/en/context-window
- https://academy.claude.com/tutorials/parametric-memory-and-context
- https://code.claude.com/docs/en/prompt-caching
- Building AI Agents in Pure Python - Beginner Course
  - https://www.youtube.com/watch?v=c9AnqCeyxbI